

DEVELOPMENT OF A TAXONOMY OF RESPONSE TYPES BASED ON RUN-TIME DATA ANALYSIS

Master Thesis

submitted: 31.January 2025

by: Bilegjargal Battulga

bilegjargal.battulga@student.uni-mannheim.de

born June 8th, 1997

in Darkhan, Mongolia

Student ID Number: 1657656

Supervisor: Dr. Marcus Kessel

Chair of Software Engineering
School of Business Informatics and Mathematics
University of Mannheim
D – 68159 Mannheim
Phone: +49 621-181-3912, Fax +49 621-181-3909
Internet: <http://swt.informatik.uni-mannheim.de>

Abstract

The increasing adoption of Large Language Models (LLMs) in software development has raised critical questions about the functional equivalence, correctness, and behavior of their generated code. While prior research has extensively benchmarked LLM performance, there is a lack of studies focusing on categorizing the diverse responses of LLM-generated code.

This thesis addresses this gap by developing a taxonomy to classify software responses, providing deeper insights into their behavior and functional characteristics. To build this taxonomy, the methodology proposed by Usman et al. was adapted and extended to focus on software responses. Data from HumanEval and MBPP benchmarks, along with their Java translations by MultiPL-E, formed the basis of the analysis. Responses were categorized into matched and unmatched data. For matched data, relationships such as behavioral equivalence and functional similarity, contextual relationship were explored. For unmatched data, Java exceptions, Randoop exceptions, Null, NaN, and valid but diverging outputs were analyzed to identify implementation flaws, systematic testing challenges, and translation anomalies.

The study revealed significant issues in edge case handling, translation fidelity, and logical consistency in LLM-generated code. While the taxonomy provides a preliminary framework for understanding functional behavior, its scope was limited to specific functional abstractions and subsets of data due to the exploratory nature of the research.

Contents

Abstract	ii
List of Figures	vi
List of Tables	viii
List of Abbreviations	ix
1. Introduction	1
2. Background and Related Work	3
2.1. Benchmarks	3
2.1.1. HumanEval	3
2.1.2. MBPP	4
2.1.3. MultiPL-E	4
2.2. LASSO	6
2.3. Taxonomy	7
2.4. Evaluation of LLM Outputs	9
3. Building a Taxonomy	11
3.1. Methodology for Developing a Taxonomy	11
3.2. Applying the Methodology	13
4. Data Management	16
4.1. Dateset Description	16
4.2. Problems with Dataset	19
4.3. Data Preprocessing	22
4.3.1. Code Coverage Data	22
4.3.2. Procedural Data	24
4.3.3. Non-Functional Output Rows	25
4.3.4. Irrelevant Columns	25

4.3.5. Data Type	26
4.3.6. Handling Missing Values and Duplicates	28
4.3.7. Deriving Matched and Non-matching Data	29
5. Matched Data Analysis	31
5.1. Abstraction Clusters	31
5.2. Relationship Analysis	33
5.2.1. Behaviorally Equivalent Clusters	34
5.2.2. Partially Behaviorally Equivalent Clusters	36
5.2.3. Functionally Similar and Contextually Related Clusters . .	37
5.2.4. Functionally Similar Clusters	41
5.2.5. Contextually Related Clusters	43
5.2.6. Identical Abstractions	45
5.2.7. Unrelated Clusters	45
6. Non-matching Data Analysis	49
6.1. Java Exceptions	49
6.1.1. Analysis of java.lang.IllegalMonitorStateException	54
6.1.2. Analysis of java.lang.ClassCastException	55
6.1.3. Analysis of java.lang.IndexOutOfBoundsException	57
6.1.4. Analysis of "java.lang.StringIndexOutOfBoundsException" . .	58
6.1.5. Analysis of "java.lang.NumberFormatException"	59
6.1.6. Analysis of "java.lang.ArrayIndexOutOfBoundsException" . .	60
6.1.7. Analysis of "java.lang.StackOverflowError"	61
6.2. Analysis of Randoop exceptions	62
6.3. Analysis of Null rows	64
6.4. Analysis of Nan rows	65
6.5. Valid Non-matching Dataset	67
6.5.1. Analysis of HumanEval_87_get_row	69
6.5.2. Analysis of mbpp_787_text_match_three	71
6.5.3. Analysis of mbpp_737_check_str	72
6.5.4. Analysis of mbpp_91_find_substring	73
6.5.5. Analysis of mbpp_605_prime_num	74
6.5.6. Analysis of mbpp_798_sum	75
6.5.7. Analysis of mbpp_127_multiply_int	76

7. Final Results	77
7.1. Complete Taxonomy Overview	77
7.2. Results for Matched Data Analysis	78
7.3. Results for Non-matching Data Analysis	80
7.3.1. Exception and Errors	80
7.3.2. Valid Non-matching Outputs	81
8. Discussion	83
8.1. Limitations	83
8.2. Future Work	85
9. Conclusion	86
Bibliography	vii
Appendix	xi
Appendix for Chapter 5	xii
Appendix for Chapter 6	xiii
A.1. Appendix for Section 6.1	xiii
B.2. Appendix for Section 6.2	xvi
C.3. Appendix for Section 6.7	xviii

List of Figures

4.1. Comparison of rows, Sheet IDs, and System IDs between HumanEval and MBPP.	17
4.2. Workflow of Data Preprocessing.	22
4.3. Comparison of 8 cases.	23
4.4. Example of "op" entry: "HumanEval_58_Common".	24
4.5. Example of "seq" Entry: "HumanEval_58_Common".	24
4.6. JUnit Assertions in "exseq": "HumanEval_58_Common".	25
4.7. Comparison of rows, Sheet IDs, and System IDs between HumanEval and MBPP after preprocessing steps.	29
4.8. Output row distribution across datasets.	30
5.1. Comparison of rows, Sheet IDs, and System IDs between HumanEval and MBPP benchmarks in matched dataset.	32
5.2. Clusters of "mbpp_127_multiply_int" and "mbpp_458_rectangle_area" with different input-output pairs.	32
5.3. Broadly connected clusters	39
6.1. Categorization of java exceptions by frequency.	50
6.2. Number of abstractions affected by each java exception.	51
6.3. Java exceptions across System IDs.	52
6.4. Most Frequently Occurring java exceptions and Their Distribution Across Abstractions.	53
6.5. Relationship between the number of unique SYSTEMIDs and number of unique inputs for each ABSTRACTIONID: Randoop exceptions.	62
6.6. Relationship between the number of unique SYSTEMIDs and number of unique inputs for each abstraction: Null rows.	65
6.7. Relationship between the number of unique System IDs and number of unique inputs for each Abstraction ID: Nan rows.	66
6.8. Distinct output counts per functional abstraction.	68

7.1. Taxonomy of software responses.	78
--	----

List of Tables

2.1. Comparison of HumanEval and MBPP benchmarks.	5
2.2. Comparison of taxonomy studies in SE.	8
2.3. Evaluation of LLM outputs.	10
3.1. Example of applying the methodology [30].	12
3.2. Application of the methodology to this thesis.	15
4.1. Description of Key Columns in the Dataset [19].	19
5.1. Partially Behaviorally Equivalent Clusters.	36
5.2. Functionally similar clusters.	42
7.1. Cluster Analysis Results for Matched Data.	79
7.2. Summary of Errors and Exceptions in the Non-Matching Dataset.	80
A1. Categorization of abstraction clusters by behavioral relationships	xii

List of Abbreviations

AI	Artificial Intelligence
GPT	Generative Pre-training Transformer
LASSO	Large-Scale software Observatory
LLM	Large Language Model
MBPP	Mostly Basic Python Problems
RQ	Research Question
SE	Software Engineering
SRM	Stimuli-Response Matrices
SWEBOK	Software Engineering Body of Knowledge
TDSE	Test Driven Software Engineering

1. Introduction

Software testing is any activity aimed at evaluating an attribute or capability of a program or system and determining that it meets its required results [11]. It is a critical component of the software development lifecycle, ensuring that the developed system operates as intended and meets user requirements. For the last few years, Large Language Models (LLMs) like OpenAI’s Codex [21], Google’s PaLM [10], and others have revolutionized software testing by automating and enhancing various aspects of the process. They are utilized in many software testing tasks, such as automated test case generation, code review and bug detection, documentation generation, refactoring and optimization, and even for simulating test environments. Although they have shown great success, some challenges exist, such as bias and limitations, scalability, functional correctness, and context understanding.

Researchers have explored multiple approaches to improve the performance of LLMs, ranging from refining inputs, optimizing prompts and systematically evaluating outputs [33] [23] [17]. However, there is currently no existing taxonomy designed to classify the diverse response generated by LLMs. Without such a taxonomy, understanding the behavioral patterns and functional relationships in LLM-generated outputs remains a complex and arduous task. Therefore, this thesis aims to address this gap by developing a taxonomy to categorize software responses, providing deeper insights into their behavior and functional characteristics. In order to build the taxonomy of software responses, the methodology proposed in [30] was adapted.

The data analyzed in this thesis is extracted from LASSO [12], large-scale software observatorium platform, that replicated and extended the methodology of Multipl-E [6], which translated HumanEval [7] and MBPP [2] coding benchmarks into 18 programming languages and tested multiple LLMs. LASSO focused on the Java version of these benchmarks, replicating, reproducing, and repurposing the data to facilitate in-depth analysis of Java-based code execution and behavior.

In order to achieve the goal of this thesis, following three Research Questions (RQ) are specified:

- RQ1: How can a taxonomy be developed to systematically categorize software responses generated by LLMs?
- RQ2: What relationships and patterns exist between functional abstractions, test behaviors, and response types?
- RQ3: What insights can be gained from analyzing the errors, mismatches, and behavioral patterns in LLM-generated outputs?

The rest of this thesis is structured as follows. Chapter 2 provides a comprehensive review of the foundational concepts and related research. Then, a concept regarding the RQ 1 is described in Chapter 3. Next, chapter 4 outlines the data preparation process, by filtering out irrelevant columns and rows. It also derives matching and non-matching datasets, which are specifically designed to address the research questions of this thesis. Subsequently, a concept regarding RQ 2 and RQ 3 are described in Chapters 5 and 6, which focus on the analysis and clustering of matching and non-matching data. Furthermore, final results are presented in the Chapter 7. Finally, discussion and conclusion are drawn in Chapters 8 and 9, respectively.

2. Background and Related Work

This chapter provides background information and related research essential to this thesis. It begins with an overview of benchmarking techniques for evaluating LLM-generated code, focusing on HumanEval, MBPP, and MultiPL-E in Section 2.1. Then, in Section 2.2, it discusses LASSO and its role in replicating and extending these benchmarks. Following this, Section 2.3 reviews existing taxonomy research related to software responses, highlighting gaps in classification. Finally, Section 2.4 explores research on LLM output evaluation methods.

2.1. Benchmarks

One of the most widely adopted approaches for evaluating LLM-generated code is benchmarking. Benchmarks provide a standardized way to assess the accuracy and functionality of the outputs, allowing researchers to identify strengths, weaknesses, and areas for improvement. This thesis leverages data derived from benchmarks that focus on code generation and functional correctness, namely HumanEval and MBPP, translated and extended through the MultiPL-E framework. Each of these datasets can be found in [20], [8], and [18], respectively.

2.1.1. HumanEval

The HumanEval benchmark was introduced by OpenAI as part of their research on Codex, a GPT model fine-tuned on programming data [7]. It consists of 164 hand-crafted Python programming problems, each accompanied by function signatures, docstrings, and hidden unit tests. The benchmark evaluates the functional correctness of generated code, prioritizing accurate and executable solutions over surface-level textual similarity. Pass rates, calculated based on the ability of generated code to pass unit tests, serve as the primary metric for assessing performance.

HumanEval emphasizes functional correctness because it aligns with real-world expectations in software testing, where the accuracy of outputs is critical. By hiding unit tests during the code generation phase, HumanEval ensures that LLMs generate solutions based on problem understanding rather than reverse-engineering the tests. This benchmark has been pivotal in identifying the strengths and limitations of LLMs in generating accurate, executable code, making it a foundational dataset for studying model behavior in Software Engineering (SE) contexts [7].

2.1.2. MBPP

MBPP, introduced in [2], is another influential benchmark that focuses on basic coding problems. It consists of 974 problems covering a range of challenges, from simple algorithms to tasks requiring deeper logical reasoning. Each problem includes a description, input-output examples, and assertions to test correctness. Its diversity, drawn from educational programming exercises, makes it an ideal dataset for evaluating general coding capabilities. Its problems provide insights into how well LLMs generalize to a wide range of challenges, complementing HumanEval in assessing model outputs.

An important aspect of MBPP is its inclusion of test cases as part of the input prompt, enabling models to generate solutions tailored to those specific tests. This methodology allows the benchmark to evaluate how well models can adhere to and pass given test cases, serving as an indicator of their coding accuracy and ability to follow constraints [2].

2.1.3. MultiPL-E

MultiPL-E, introduced in [6], addresses the monolingual bias of traditional benchmarks like HumanEval and MBPP by translating them into 18 additional programming languages. This extension evaluates LLMs across a broad range of programming paradigms, type systems (static and dynamic), and language-specific features. MultiPL-E employs automated compilers to translate function signatures, comments, and unit tests while preserving the semantic integrity of the original tasks.

This multilingual benchmarking approach highlights the strengths and weaknesses of LLMs across different programming languages, offering a more comprehensive understanding of their capabilities. The inclusion of Java benchmarks, which is central to this thesis, allows for the exploration of functional abstractions and response behaviors in a statically typed language.

To align MBPP with HumanEval’s, MultiPL-E refined the benchmark by separating the test cases from the input prompt. This adjustment ensures unbiased evaluation, requiring models to generate solutions based solely on problem descriptions and function signatures, without relying on embedded test case hints [6]. Table 2.1 compares the characteristics of HumanEval and MBPP benchmarks as extended through MultiPL-E.

Aspect	HumanEval	MBPP
Purpose	Evaluates functional correctness of Python code generation by LLMs.	Evaluates functional correctness of Python code generation for common programming problems.
Number of Problems	164	974
Problem Categories	General-purpose coding problems.	Categorized by difficulty: basic, intermediate, and advanced.
Problem Format	Includes a natural language problem description and function signature.	Includes a natural language problem description, function signature, and test cases embedded in the prompt.
Test Cases	Separate from the input prompt and provided as a list for evaluation only.	Embedded as part of the input prompt, guiding the model to generate solutions that adhere to the test cases.
Evaluation Methodology	Models are tasked to generate function implementations that pass the test cases.	Models are tasked to generate function implementations that pass the embedded test cases.
Evaluation Metrics	Pass@k metric: percentage of solutions passing the test cases within k samples (e.g., pass@1, pass@10, pass@100).	Pass rate for generated code against the embedded test cases.
Dataset Creation	Manually curated problems by experts.	Curated problems from real-world Python programming scenarios.
Language	Python	Python
Focus	Testing correctness of standalone Python functions in isolated contexts.	Testing correctness of Python solutions across various real-world programming challenges.
MultiPL-E Translation	Translated into 18 programming languages (including Java, C++, TypeScript) by MultiPL-E.	Translated into 18 programming languages by MultiPL-E with refinement (test cases separated from the prompt).

Table 2.1.: Comparison of HumanEval and MBPP benchmarks.

2.2. LASSO

In [12], authors introduce the LASSO platform, which provides a comprehensive framework for conducting, replicating, and extending experiments in test-driven software experimentation (TDSE). It addresses key challenges in software testing by introducing innovative data structures to represent and analyze software behaviors.

LASSO’s primary goal is to promote transparency, reproducibility, and scalability in software testing by offering tools and data structures that align with open science principles. These include clear representations of test cases and results, systematic methods for storing and analyzing stimuli and responses, and frameworks for defining high-level experimental pipelines [12].

Among the core data structures introduced by LASSO are the Sequence Sheet Notation, Stimuli-Response Matrices (SRMs), and the Stimulus-Response Hypercube. Sequence Sheet Notation standardizes the representation of operation sequences or test cases applied to software components, capturing the order of stimuli and their associated responses. SRMs organize inputs (stimuli) and their corresponding outputs (responses) in a matrix format, enabling systematic comparisons across various tests and software components. The Stimulus-Response Hypercube extends this concept by incorporating additional dimensions such as time, software variants, and execution environments, offering a comprehensive, multi-dimensional perspective on testing data. These data structures collectively facilitate the detailed analysis and interpretation of TDSE data [12].

LASSO integrates and extends the MultiPL-E benchmarks, which translated Python-based HumanEval and MBPP tasks into 18 programming languages, including Java. LASSO focused specifically on the Java benchmarks, addressing challenges unique to statically typed languages while replicating and enhancing MultiPL-E’s experimental design. This involved:

- Replication: Reproducing MultiPL-E’s benchmarking experiments to assess LLM performance on Java-translated benchmarks.
- Reproduction: Improving upon the experimental setup to ensure accurate representation of Java-specific behaviors and challenges, such as serialization and automated test generation.

- Repurposing: Utilizing the benchmark data to explore new research questions, including the development of a taxonomy for categorizing software responses.

This thesis primarily utilizes the SRM_full dataset, which is derived from LASSO's experiments with MultiPL-E benchmarks. The dataset provides a rich foundation for analyzing software responses, capturing the relationships between stimuli and responses, and exploring patterns in test behaviors. By leveraging its advanced data structures, this thesis aims to systematically analyze LLM-generated outputs, contributing to the development of a taxonomy that categorizes software responses based on their behaviors and characteristics.

2.3. Taxonomy

Taxonomy is a scheme that partitions a body of knowledge and defines the relationships among the pieces. It is used for classifying and understanding the body of knowledge [9]. It provides a structured framework that facilitates organization, comparison, and communication of complex information. Taxonomies are widely used across various disciplines, including biology, medicine, business, and computer science, to bring order and clarity to vast and complex datasets. In SE, taxonomies are applied to systematize various aspects of the field, from defects and testing methodologies to system behaviors.

At the time of writing this thesis, no existing taxonomies in SE specifically classify software responses. To explore related work, a literature search was conducted through Google Scholar using keywords such as "taxonomy of software responses", "software output classification", and "software failure taxonomy". Table 2.2 summarizes the key studies identified during this search, highlighting their objectives, focus areas, and contributions.

In addition, [30] provides an extensive characterization of research on SE taxonomies. The study analyzed 270 papers and categorized taxonomies based on the SE Body of Knowledge (SWEBOK) areas, finding that most taxonomies fall within software construction (19.55%), design (19.55%), requirements (15.50%), and maintenance (11.81%). This classification highlights the absence of taxonomies dedicated to software outputs, reinforcing the need for a structured approach to categorizing response behaviors in software testing.

Reference	Research Objective	Key Contributions and Findings
Agile Development		
[29]	Develop a taxonomy to systematically organize knowledge on effort estimation in Agile Software Development (ASD) and support both research and practice.	Proposes a four-dimensional taxonomy covering estimation context, estimation techniques, effort predictors, and effort estimates. The taxonomy highlights common estimation techniques such as story points and planning poker, identifies key effort predictors, and classifies estimation activities within Agile workflows. Findings indicate that many studies fail to report complete estimation-related information, limiting comparability. The taxonomy facilitates structured reporting and improves understanding of effort estimation in ASD.
Global Software Engineering		
[26]	Develop a standardized terminology and taxonomy to unify global software engineering research and practice.	Finds that GSE terminology is inconsistent, with many synonyms for the same concepts, ambiguous definitions, and inconsistent usage across studies. Conducts a Delphi-inspired study with GSE experts to develop a standardized glossary and taxonomy for sourcing strategies. The taxonomy categorizes GSE-related terms based on generalization-specialization relationships, improving clarity and coherence in research reporting.
[4]	Develop a specialized taxonomy for effort estimation in GSE, based on existing classifications and empirical findings.	Extends [26] by incorporating three new dimensions: estimation process role, estimation stage, and estimation process architectural model. Identifies factors specific to effort estimation in GSE projects, such as geographical, temporal, and socio-cultural influences. Findings highlight that the lack of a structured classification in GSE leads to inconsistencies in effort estimation reporting, making comparisons difficult.
[5]	Extend the existing GSE taxonomy by incorporating additional dimensions relevant to distributed software development.	Expands [26] by adding seven new dimensions: setting, software process type, software process distance, power distance, uncertainty avoidance, language distance, and communication model. Benchmarks the extended taxonomy against existing taxonomies and validates it through classification of eight real-world GSE projects. The findings demonstrate that the new dimensions improve the ability to describe and compare GSE contexts, enhancing the clarity, synthesis, and applicability of research in distributed software engineering.
Software Evolution		
[15]	Develop a taxonomy characterizing software evolution based on mechanisms of change rather than purpose.	Proposes a taxonomy structured around four dimensions: **Temporal Properties (When), Object of Change (Where), System Properties (What), and Change Support (How)**. Unlike prior taxonomies, this framework focuses on the technical aspects of software evolution. It was validated by analyzing three tools (Refactoring Browser, CVS, and eLiza self-managing servers), demonstrating its utility in comparing tools, identifying strengths/weaknesses, and guiding improvements.
Requirements Engineering		
[13]	Develop a taxonomy to classify requirements changes in Agile software development.	Identifies key characteristics of requirements changes (RCs) in Agile projects, including their types, forms, reasons, sources, carriers, and emergence events. The taxonomy is derived from a mixed-methods study involving interviews, a literature review, and a global survey of Agile practitioners. Findings highlight that RCs originate from multiple stakeholders and occur frequently due to evolving project needs. The taxonomy provides a structured guide to help Agile teams better understand, manage, and respond to RCs effectively.
[24]	Develop a general taxonomy to classify and identify requirement interactions in software systems.	Proposes a four-layered taxonomy that systematically categorizes requirement interactions into 6 main categories, 17 subcategories, 29 interaction types, and 29 interaction scenarios, each with corresponding detection guidelines. The taxonomy is domain-independent, making it applicable across different software domains. It enhances the understanding of requirement interactions by providing real-life examples and structured detection methodologies. Comparative analysis with existing taxonomies demonstrates its comprehensiveness, addressing prior limitations while introducing new interaction types not covered in previous work.
Software Process Improvement		
[3]	Identify key success factors for the effective deployment of software processes.	Proposes a taxonomy that classifies critical success factors (CSFs) into three hierarchical levels: categories, subcategories, and items. The taxonomy was developed using a systematic literature review and industry experiences. Key CSFs identified include technical, organizational, and human aspects, with a strong emphasis on the role of people in process adoption. The study highlights that organizations following a people-centered strategy experience smoother process deployment. The taxonomy also helps standardize terminology, assess deployment risks, and provide structured guidelines for improving process implementation success.

Table 2.2.: Comparison of taxonomy studies in SE.

2.4. Evaluation of LLM Outputs

As introduced in chapter 1, evaluating the outputs of LLMs for code generation has become a critical area of research. Various approaches have been developed to assess the functional correctness, scalability, and utility of LLM-generated code, with a significant emphasis on benchmarking frameworks, prompt engineering, and dynamic/static analysis. These methodologies are essential for understanding the strengths and limitations of LLMs in generating code and improving their performance.

The existing works can broadly be categorized into three main approaches:

- **Benchmarking Frameworks:** These focus on evaluating LLM outputs using standardized tasks and datasets to assess functional correctness and domain-specific performance. Examples include frameworks like Multi-CodeBench and HumanEvo, which provide detailed, task-specific evaluations across programming languages and development domains.
- **Prompt Engineering:** This category examines the impact of crafting effective prompts on LLM performance. Studies such as those by Murr et al. and Jiang et al. explore techniques like zero-shot, few-shot, chain-of-thought prompting, and evolutionary optimization to improve code generation outcomes.
- **Dynamic/Static Analysis:** These methods evaluate LLM-generated code for security, performance, and correctness by applying runtime and compile-time analysis. Examples include tools like AutoSafeCoder and CodeJudge, which combine fuzz testing and semantic analysis for enhanced reliability.

Table 2.3 presents a comparison of these methodologies, summarizing their focus areas, key features, and contributions to the evaluation of LLM-generated code.

Reference	Research Objective	Key Contributions and Findings
Benchmarking Frameworks		
[32]	Develop MultiCodeBench to assess LLM performance across 12 domains and 15 programming languages.	Introduces a benchmark with 2,400 tasks, ensuring high-quality and domain-specific challenges. Analyzes LLMs' strengths and weaknesses across different domains, identifying areas where models struggle in domain-specific code generation.
[33]	Address the overlooked issue of evolving software projects in LLM evaluations.	Introduces HumanEvo, an evolving-aware repository-level benchmark. Demonstrates that ignoring software evolution inflates LLM performance by 10–61%. Provides actionable suggestions for more realistic LLM code evaluations.
[28]	Introduce an LLM-based framework to evaluate code correctness without test cases.	Shows that guiding LLMs through "slow thinking" enhances evaluation accuracy. CODEJUDGE outperforms state-of-the-art (SOTA) evaluation methods even when using smaller models. Provides insights into LLM-based evaluation for code correctness.
Prompt Engineering		
[23]	Establish a taxonomy for systematically designing and evaluating LLM prompts.	Proposes TELeR, a general taxonomy for structuring prompts. Enhances comparability across LLM studies by categorizing prompts effectively. Encourages researchers to standardize prompt reporting for meaningful benchmarking.
[31]	Document reusable prompt patterns to improve interactions with LLMs.	Introduces a catalog of prompt patterns similar to software design patterns. Demonstrates how combining prompt patterns can enhance LLM outputs. Highlights challenges and future directions in refining prompt engineering.
[27]	Optimize prompt engineering for code generation using an evolutionary algorithm.	Proposes EPiC, a search-based approach that evolves prompts to improve LLM-generated code. Achieves 93.5% pass@1 on HumanEval and 79% on MBPP with lower cost. Demonstrates effectiveness in refining LLM-generated prompts.
[16]	Assess how different prompt styles impact LLM-generated code.	Finds that providing test cases enhances code accuracy. GPT-3.5 and GPT-4 perform well across various prompts, while Claude-2 benefits from specificity. Ambiguity negatively affects code correctness.
[22]	Provide a structured overview of prompt engineering methods and applications.	Categorizes 29 prompt techniques by functionality, application areas, and models. Highlights strengths and limitations of different prompt strategies. Identifies research gaps and ethical considerations in prompt engineering.
[1]	Develop an interactive toolkit for experimenting with LLM prompts and responses.	Introduces ChainForge, an open-source system for testing LLMs across prompt variations. Identifies three modes of prompt engineering: exploration, evaluation, and refinement. Supports real-world hypothesis testing and prompt optimization.
Dynamic/Static Analysis		
[25]	Assess LLMs' ability to evaluate software quality compared to static analysis tools.	Finds GPT-3.5 Turbo correlates well with SonarQube's maintainability scores. GPT-4o produces inconsistent quality evaluations. Identifies limitations like cost, output variability, and LLM constraints in assessing structural code properties.
[17]	Improve security in LLM-generated code using a multi-agent framework.	Introduces a system with coding, static analysis, and fuzz testing agents. Reduces security vulnerabilities by 13% without sacrificing functionality. Highlights benefits of integrating static and dynamic analysis in LLM-driven development.

Table 2.3.: Evaluation of LLM outputs.

3. Building a Taxonomy

This chapter details the process of developing a taxonomy to analyze software testing responses, particularly outputs generated by LLMs for coding problems. First, Section 3.1 introduces the structured framework adopted for taxonomy development that guides the systematic classification process. Then, this methodology is applied in Section 3.2.

3.1. Methodology for Developing a Taxonomy

The taxonomy aims to classify outputs, identify behavioral relationships among functional abstractions, and diagnose errors or mismatches. A structured methodology is adapted from the four-phase framework proposed in [30], to ensure a systematic approach. The four phases are: Planning, Identification and Extraction, Design and Construction, and Validation, and further divided into 13 activities, identified from B1 to B13. Each activity provides a clear and systematic step toward creating a robust taxonomy. The authors describe these phases as follows:

- Planning: has 6 activities that define the taxonomy’s context and initial setting.
- Identification and Extraction: comprises 2 activities for extracting and controlling the terms associated with the taxonomy.
- Design and Construction: comprises 4 activities to support identification and description of the dimensions, categories and relationships of the new taxonomy.
- Validation: To ensure that the selected subject matter is clearly, concisely and thoroughly classified, it is necessary to validate the new taxonomy.

To demonstrate how this methodology can be applied, they illustrated it with an example of taxonomy in [14], which focused on classifying hypermedia and web

application size metrics. The Table 3.1 includes the descriptions of each activity (B1 to B13) and their application to [14].

Phase	Id	Description of the Ids	Application to Mendes et al.'s Taxonomy
Planning	B1	Define SE knowledge area	The SE knowledge area associated with the taxonomy is <i>SE project management</i> .
	B2	Define objectives of the taxonomy	The main objective of the taxonomy is to classify hypermedia and web application size metrics reported in the literature.
	B3	Describe the subject matter to be classified	The subject matter is hypermedia and web application size metrics.
	B4	Select the classification structure type.	A <i>facet-based classification structure</i> was selected for the taxonomy.
	B5	Select classification procedure type.	The classification procedure used is <i>qualitative</i> .
	B6	Identify the sources of information	The taxonomy is based on software measurement concepts drawn from literature on software size metrics and measurement.
Identification and Extraction	B7	Extract all terms	All dimensions and categories were extracted from the literature.
	B8	Perform terminology control	Terminology control was not necessary as the dimensions and categories resulted from an aggregation of concepts.
Design and Construction	B9	Identify and describe taxonomy dimensions.	Eight dimensions were identified, including harvesting time, metric foundation, class, and entity.
	B10	Identify and describe categories	Twenty-five categories were defined across the dimensions, such as harvesting time (early/late) and metric foundation.
	B11	Identify and describe relationships	The classification structure is facet-based; the dimensions and categories are <i>orthogonal</i> .
	B12	Define the guidelines for using and updating the taxonomy	No explicit guidelines were proposed, but the taxonomy was applied to classify metrics, giving insight into its usage.
Validation	B13	Validate the taxonomy	The taxonomy was validated by ensuring orthogonality, but no benchmarking was carried out.

Table 3.1.: Example of applying the methodology [30].

3.2. Applying the Methodology

In this thesis, a facet-based classification structure is employed. Faceted analysis allows multiple perspectives (facets) to classify complex entities, ensuring flexibility and adaptability. According to authors: "Faceted analysis leads to taxonomies whose subject matters are classified using multiple perspectives (facets). Each facet is independent and can have its own classes, which enable facet-based taxonomies to be easily adapted so they can evolve smoothly over time" [30].

The top-level dimensions (facets) for this taxonomy are: Matching Outputs and Non-matching Outputs, reflecting the nature of the given dataset, which includes both oracle outputs and original outputs. This division facilitates focused exploration of distinct behavioral and error-related patterns.

Matching Outputs refer to outputs that align with oracle results. These outputs are further categorized into three main groups:

- Identical Outputs: Functional abstractions exhibiting identical input-output behavior, indicating complete equivalence in behavior between the original and oracle outputs.
- Overlaps (Unrelated): Outputs that overlap in certain aspects but lack any meaningful relationships or shared functionality.
- Relationships: Clusters that include Behaviorally Equivalent, Partially Behaviorally Equivalent, Functionally Similar, and Contextually Related outputs. These relationships highlight varying degrees of functional or contextual alignment among abstractions.

Non-matching Outputs represent outputs that differ from oracle results. These are categorized into two main groups: "Valid Non-Matching Outputs", which are outputs that differ from oracle results but remain valid, and "Error Types", which include various errors or exceptions. The Error Types are further divided into four groups:

- Java Exceptions: Representing runtime errors encountered during code execution.
- Randoop Timeouts: Representing errors caused by Randoop test timeouts.
- Null Rows: Referring to outputs with null values.

- NaNs: Denoting outputs classified as "Not a Number" values, which in this dataset have a value of None.

In addition, the taxonomy employs both qualitative and quantitative classification procedures. Qualitative methods include behavioral analysis to define categories such as "Behaviorally Equivalent" or "Contextually Related" and manual clustering of matched outputs based on input-output overlaps. Quantitative methods include automated clustering using input-output mappings and error type grouping in non-matching outputs for large-scale analysis.

Table 3.2 provides an overview of how each phase (Planning, Identification and Extraction, Design and Construction, and Validation) and its corresponding activities (B1 to B13) have been tailored to this thesis. Additionally, it includes references to the relevant chapters and sections where each activity is discussed in detail.

Phase	ID	Context in This Thesis	Addressed in Chapter/Section
Planning	B1	The knowledge area addresses software testing responses, focusing on outputs generated by LLMs tested against translated HumanEval and MBPP benchmarks.	Chapter 1 - Discussed as part of the research context and objectives
	B2	The taxonomy aims to classify software testing responses into matched and non-matching outputs, supporting behavioral analysis and error categorization.	Chapter 1 - Outlined in the research objectives and scope
	B3	The subject matter includes functional abstractions and their responses from the SRM_full dataset, derived from source codetesting.	Section 4.1 - Details the SRM_full dataset and its components
	B4	A facet-based classification structure is used, with two top-level dimensions: Matched Outputs and non-matching Outputs.	Chapter 3 - Introduced and described in this chapter
	B5	A combination of qualitative (manual clustering) and quantitative (input-output mapping) methods is used for classification.	Chapter 3 - Methodology explained; implementation in Chapters 5 and Chapter 6
	B6	The SRM_full dataset, exported from LASSO, serves as the primary source, including details about test cases, outputs, and input-output mappings.	Chapter 4, Section 4.1 and Section 4.2
Identification and Extraction	B7	Key terms were derived from the SRM_full dataset.	Chapter 4, Section 4.1
	B8	Terms like Behaviorally Equivalent, Partially Equivalent, and Contextually Related were introduced to ensure clarity in describing relationships.	Chapter 5, Section 5.2 (with subcategories)
Design and Construction	B9	Dimensions were defined as Matched Outputs (aligned with oracle outputs) and non-matching Outputs (differed from oracle outputs).	Chapter 3 - Overview; detailed implementation in Chapter 5 and Chapter 6
	B10	Categories for Matched Outputs and non-matching Outputs were defined, with further subcategories.	Chapter 5, subsections under Section 5.2
	B11	Relationships between categories were analyzed for behavioral equivalence and error causes.	Chapter 5, Section 5.2 and Chapter 6
	B12	Guidelines for categorizing new data and refining taxonomy were developed for future applications.	Chapter 8, Section 8.2
Validation	B13	Validation was limited to representative examples due to dataset size and time constraints; future efforts will focus on broader validation.	Chapter 8, Section 8.1

Table 3.2.: Application of the methodology to this thesis.

4. Data Management

The chapter begins by describing the SRM_full and Solr_class datasets, emphasizing their structure, origin, and significance for analyzing functional abstractions and response type in the Section 4.1. Then, the Section 4.2 highlights the problems the authors from both MultiPL-E and LASSO had, along with their potential impact on this analysis. Finally, the Section 4.3 outlines the preprocessing steps taken, including removal of irrelevant data, handling missing values and duplicates, and deriving matching and non-matching datasets.

4.1. Datasets Description

This thesis uses two datasets: SRM_full and Solr_class. The SRM_full dataset will be used as the primary data for this thesis, providing essential information for analyzing functional abstractions, test behaviors, and response types. It contains the data that was generated and tested using MultiPL-E [6], which translated coding benchmarks, including HumanEval [7] and MBPP [2], into 18 different programming languages and evaluated the performance of multiple LLMs. Further, the Java version of HumanEval was imported into LASSO [12] for replication, reproduction and repurposing, as LASSO focuses on the execution and analysis of code written in Java (see Section 2). While the paper focuses explicitly on the Java version of HumanEval (referred to as HumanEval-J) as the running example for demonstrating LASSO’s capabilities, the "SRM_full" dataset includes data from MBPP as well [12].

The column names of SRM_full dataset are written in uppercase and formatted as COLUMNNAME (e.g., SHEETID, SYSTEMID, ABSTRACTIONID) to maintain consistency with the original dataset. When referring to these terms in a general context rather than as specific column names, they are written in standard title case (e.g., Sheet ID, System ID, Abstraction ID) for readability.

The SRM_full dataset contains in total 16 columns and almost 39 million rows, capturing the detailed observations of inputs, outputs, and execution contexts for tests conducted on source codes generated by various LLMs. This dataset includes 1719979 unique Sheet IDs (SHEETID), each representing one of 26 unique test names applied to one of the 304055 unique source codes (identified by column SYSTEMID) and corresponding to 446 unique Abstraction IDs (ABSTRACTIONID), which represent the functional abstractions (i.e., coding problems) being tested. These include 158 unique Abstraction IDs for HumanEval tasks and 288 unique Abstraction IDs for MBPP tasks. Figure 4.1 shows the comparison of the total number of rows, unique Sheet IDs, and unique SYSTEM IDs between the HumanEval and MBPP benchmarks, highlighting the distribution and scale of these datasets.

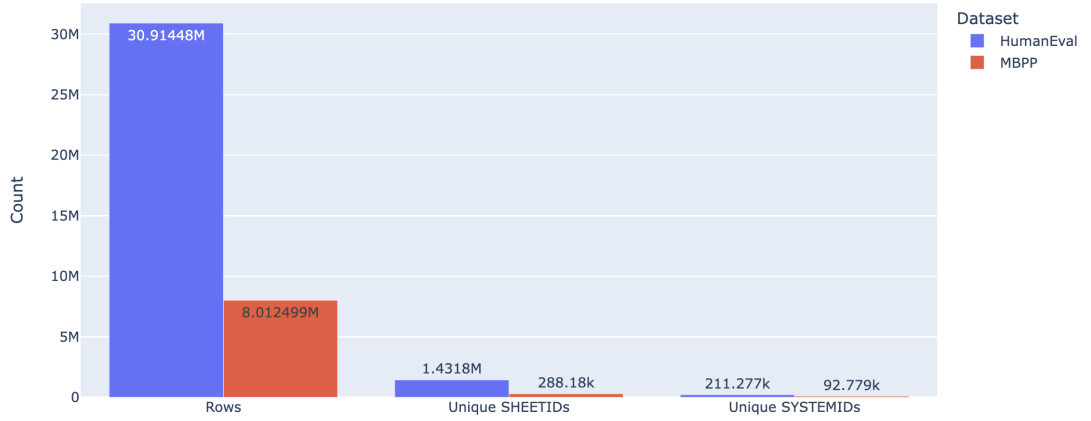


Figure 4.1.: Comparison of rows, Sheet IDs, and System IDs between HumanEval and MBPP.

The detailed description of each column is provided below, based on the original documentation from [19]. Additional observations have been incorporated based on data exploration and preprocessing steps to enhance understanding of column behavior and dataset characteristics, as detailed below:

- **EXECUTIONID**: Identifies the LSL script execution ID. Unique values are: "821f389c-5b19-4da3-b7cf-2fe4610738b0" and "31a03823-2a33-4cce-8581-44761ba37e7a".
- **ABSTRACTIONID**: Represents the functional abstraction being tested. There are 446 unique "ABSTRACTIONID", including 158 abstractions for

HumanEval tasks and 288 abstractions for MBPP tasks.

- ACTIONID: Refers to LSL action ID. For this dataset, it is always "execute".
- ARENAID: Indicates the arena test driver. Unique values are: "jacoco" (code measurements) or "execute" (normal test execution).
- SHEETID: Represents a test name applied to a source code. It can be either a compound of test name and "SYSTEMID" (e.g., "Problem_1_54fb5fa3-9eec-4011-8224-dd3030db48b8", where "Problem_1" is a test name and "54fb5fa3-9eec-4011-8224-dd3030db48b8" is "SYSTEMID") or "all". There are in total 26 unique test names, and 1719979 unique sheet ids, including "all".
- SYSTEMID: Identifies the source code being tested. It could be either a specific id of a source code (e.g., "54fb5fa3-9eec-4011-8224-dd3030db48b8") or "oracle". There are in total 304055 unique values, including "oracle".
- VARIANTID: Differentiates between the original data or oracle. Depicts the values of the columns SYSTEMID.
- ADAPTERID: Represents the adapter applied for testing. Similar to VARIANTID. It is "0" for original data and "oracle" for oracle data.
- X, Y: Represent the horizontal and vertical coordinates in the sequence sheet of LASSO. (X=0, Y) refers to output values. (X=1, Y) refers to operations. (X 1, Y) refers to input values. (0,0) refers to the output value by the CREATE statement, storing the instance of the class under test.
- TYPE: Indicates the nature of the observation. Common values include: "value" (outputs), "input_value" (inputs), "op" (operation name), "seq" (java sequences), "exseq" (test sequences (JUnit like)), "jacoco*" (code measurements).
- VALUE: Contains observed outputs, exceptions, or metrics.
- RAWVALUE: Provides detailed or unprocessed observations.
- VALUETYPE: Describes the type of the value (e.g., "java.lang.String", "java.util.ArrayList" etc.)
- LASTMODIFIED: Indicates when the record was last updated.
- EXECUTIONTIME: Captures the time taken for test execution.

These columns, integrating information provided by the authors, can be grouped into categories, such as arena execution, implementations, sequence sheets, and observations. The Table 4.1 summarizes the corresponding columns for each category. In addition, the columns LASTMODIFIED and EXECUTIONTIME can be categorized as timing and updates.

Category	Column Names	Description
Identifying Arena Executions	[EXECUTIONID, ACTIONID, ARENAID]	Defines instances of arena test drivers.
Identifying Implementations	[SYSTEMID, ADAPTERID, VARIANTID]	Describes the code artifact or source code being tested.
Identifying Sequence Sheets	[SHEETID, X, Y]	Represents test sheets applied to a source code.
Identifying Observations	[VALUE, RAWVALUE, VALUETYPE, TYPE]	Captures test responses.
Identifying Timing and Updates	[LASTMODIFIED, EXECUTIONTIME]	Tracks metadata about the test execution.

Table 4.1.: Description of Key Columns in the Dataset [19].

The Solr_class dataset contains 314484 rows and 70 columns, providing comprehensive information related to the source code of the source codes. However, the columns "id" and "content" are the most relevant columns for this thesis as the column "id" corresponds to the SYSTEMID in the SRM_full dataset, and the column "content" contains the actual source code of the source codes, which is crucial for analyzing their implementations. Therefore, given the scope of this thesis, the focus is limited to the columns "id" and "content", and the other columns in the Solr_class dataset are not described in detail.

4.2. Problems with Dataset

Both MultiPL-E and LASSO faced several challenges while translating and integrating Java versions of benchmarks: HumanEval and MBPP, which could have affected the validity of the SRM_full dataset. MultiPL-E's effort to translate

Python based benchmarks into multiple languages, including Java, encountered difficulties such as inconsistencies in data structures, issues with equality testing, and mismatches in inferred types [6]. LASSO, in turn, inherited many of these issues while also facing additional obstacles, including problems with serialization, automated test generation, and the creation of custom adaptations to integrate the benchmarks [12].

These limitations are an inherent part of the dataset’s construction and integration. Therefore, this thesis does not focus on resolving these issues, but acknowledges their importance in interpreting the findings accurately. Some outputs or behaviors may reflect these challenges rather than the true performance of the source codes. More detailed overview of these challenges is provided below.

Translation Challenges by MultiPL-E

Cassano et al., mentioned that they used simplified compilers to translate elements like function signatures, unit tests, and type annotations. In other words, they did not translate the function bodies because the goal of the benchmarks was to test the LLM’s abilities. They highlight several important limitations of their multilingual benchmarking approach, and although they do not specifically mention Java much, it is worth considering them [6].

One of the primary challenges was translating Python’s flexible data structures. For instance, lists in Python can contain mixed data types; therefore, they had to be converted to structures, such as to arrays for JavaScript arrays or to vectors for R, which require uniform data types. Tuples were often flattened or converted to more straightforward data types in languages that did not support them.

Equality testing was another significant issue. Python’s “==” operator, which compares the content of nested data, was not supported in many languages. For example, JavaScript requires external libraries, making the translation process even more difficult.

Next, type annotations caused additional challenges, especially because all of the MBPP benchmarks lacked them. Authors say that “Optional” or “Any” types were particularly tricky to translate into languages like Rust or Swift because they rely on algebraic data types. Inferred types were often necessary but did not always match the original intent, thus leading to potential inconsistencies.

Moreover, Python’s doctest format (>>>) and inline comments had to be adapted

for languages where this format is not idiomatic. However, the authors kept Python’s doctest format (`>>>`) consistent and prompt across all translated languages, even though it does not match the typical style of languages like Java. This decision prioritized consistency over adapting to each language’s conventions. To ensure the translations were correct, they manually reviewed selected benchmarks and used automated test suites to check the accuracy of the prompts and outputs.

Although authors handled these challenges, they acknowledge that some threats to validity still remain, such as the benchmarks do not reflect the real-world use of target languages, the sensitivity of models to prompt designs, and the limitations of probabilistic sampling methods.

Challenges in Lasso

Kessel and Atkinson, also mentioned challenges they inherited from MultiPL-E when integrating translated to Java benchmarks into their LASSO platform for test-driven software experiments. They specifically discuss that the original HumanEval benchmark is designed for Python, therefore, the translated version might evaluate Java’s real-world performance incorrectly. Also, they mention the problem with Python’s doctest, which is not supported in Java, and their sampling technique. In this study, they reused the sample software components from MultiPL-E, noting that the results would likely remain consistent [12].

On top of these inherited threats, authors mention their own challenges, including serialization and automated test generation. The serialization problem comes from Java object outputs into JSON because this process sometimes causes differences between runtime checks and serialized data, especially for complex and nested structures. In order to handle this issue, the authors used a standardized approach for serialization, which kept the key information intact while minimizing inconsistency.

The next issue arised from automated test generation. EvoSuite was used to create tests automatically, focusing on edge cases such as invalid inputs or null values. Even though this was helpful, it sometimes produced syntactically different but still correct outputs. For instance, it returned null when an empty string was expected. To address this issue, the authors carefully reviewed the generated tests to ensure they represented correct and meaningful behavior.

Lastly, many software components did not match the test interfaces used by LASSO, so a custom adaptation was needed. The authors created custom adapters to make the components work properly with tests. It required a lot of time and effort, but it allowed the components to be fully integrated into the LASSO platform.

4.3. Data Preprocessing

As a first step, the irrelevant rows and columns for this thesis were removed. These include rows with code coverage metrics, loader-related metrics, procedural data, and non-functional outputs. Then as for data consistency management, duplicates and missing values were checked. At a final step seven separate datasets were extracted. These preprocessing steps are shown in Figure 4.2

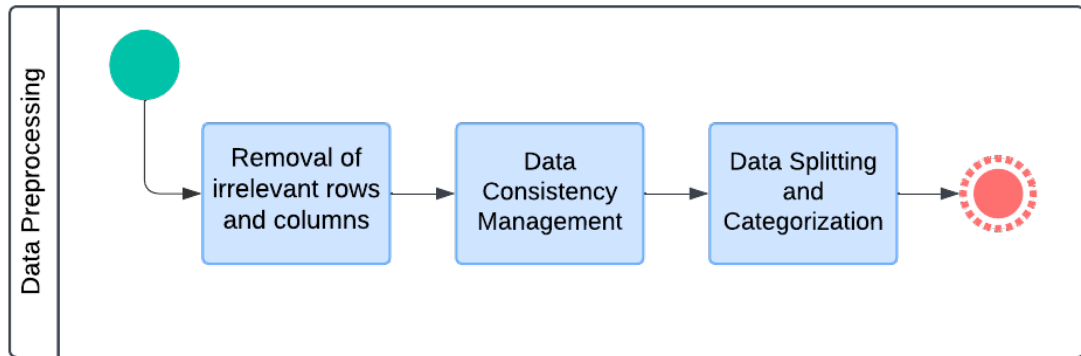


Figure 4.2.: Workflow of Data Preprocessing.

4.3.1. Code Coverage Data

As described in Section 4.1, the dataset contains nearly 39 million rows, providing detailed information about tests conducted on code candidates. These rows can be categorized into eight distinct cases based on combinations of three key attributes: ARENAID, SHEETID, and VARIANTID. Each case represents a specific type of testing information with unique characteristics and relevance to the research objectives. Figure 4.3 summarizes these cases.

Only outputs related to case 1 and case 5 were included in this thesis, as these rows were essential for comparing behaviors and identifying functional relationships.

Case Number	ARENAID	SHEETID Condition	VARIANTID	Row Count	Unique TYPE Values
1	execute	not_all	original	12600522	[seq, op, value, input_value, exseq]
2	jacoco	not_all	original	9126667	[input_value, op, value]
3	execute	all	original	607924	[loader.artifacts, loader.classes_loaded]
4	jacoco	all	original	9725280	[jacoco_class_line.missed, jacoco_class_instru...
5	execute	not_all	oracle	3439064	[value]
6	jacoco	not_all	oracle	3427524	[value]
7	execute	all	oracle	0	[]
8	jacoco	all	oracle	0	[]

Figure 4.3.: Comparison of 8 cases.

The rows related to jacoco (cases 2, 4, and 6) were excluded, as they focus on code coverage metrics that are unrelated to the research goals. In total, 22279471 rows in the dataset were identified as jacoco-related and excluded.

Next, case 3 was also excluded, as it contains 607924 rows of aggregated outputs, primarily featuring loader-related metrics. These metrics provide system-level details, but they lack the information required for analyzing input-output behavior and do not directly contribute to the goal of identifying functional equivalence.

Cases 7 and 8 were found to be empty. Case 7 (ARENAID = execute, SHEETID = all, VARIANTID = oracle) could theoretically represent the oracle version of case 3, but since case 3 only includes loader-related metrics, it is not feasible to define oracles for them. Similarly, case 8 (ARENAID = jacoco, SHEETID = all, VARIANTID = oracle) would require oracles for system-level metrics such as jacoco coverage data, which are also not applicable. Consequently, no rows exist for these cases.

After filtering the dataset, it included in total 16039586 rows:

- Case 1: 12600522 rows (inputs, outputs, and additional data such as seq, exseq, and op for original variants).
- Case 5: 3439064 rows (outputs for tests on oracle variants).

For the remainder of this thesis, case 1 will be referred to as the "original data", as it includes both original outputs, inputs and additional information. On the other hand case 5 will be referred as the "oracle outputs" as it contains only oracle outputs.

4.3.2. Procedural Data

The column TYPE provides metadata about the rows, categorizing them based on their function or role in the test environment. Unique values of the TYPE column are "input_value", "value", "op", "seq", and "exseq". Among these, rows with "op", "seq", and "exseq" were removed.

For example, a functional abstraction "HumanEval_58_common" returns sorted unique common elements for two lists. For input_1 = [2,3] and input_2 = [2,3,5,8] the expected output is a list containing common unique elements, i.e., [2,3]. Figure 4.4 shows an example of an "op" entry, which represents the operation being invoked during test execution. It indicates the name of the functional abstraction and the type of expected inputs for the method. However, this information is redundant because the column ABSTRACTIONID specifies the name of the functional abstraction and the column VALUETYPE of input rows already specify the type as "java.util.ArrayList".

```
"HumanEval_58_common.Problem.\u003cinit\u003e()"
"HumanEval_58_common.Problem.common(java.util.ArrayList, java.util.ArrayList)"
```

Figure 4.4.: Example of "op" entry: "HumanEval_58_Common".

In addition, the rows with "seq" describe sequences of operations or code snippets used during testing. As shown in Figure 4.5, they outline how the test is constructed, including object creation and method calls. However, these details are already represented in the column VALUE of input ("input_value") and output rows ("value"), which provide the necessary information about input and output types and values, making the "seq" rows redundant for this analysis.

```
HumanEval_58_common.Problem problem0 = new HumanEval_58_common.Problem();
java.util.ArrayList arrayList1 = new java.util.ArrayList<java.lang.Long>
(java.util.Arrays.asList((long) 5l, (long) 3l, (long) 2l, (long) 8l));
java.util.ArrayList arrayList2 = new java.util.ArrayList<java.lang.Long>
(java.util.Arrays.asList((long) 3l, (long) 2l));
java.util.ArrayList arrayList3 = HumanEval_58_common.Problem.common(arrayList1,
arrayList2);
```

Figure 4.5.: Example of "seq" Entry: "HumanEval_58_Common".

Similarly, the "exseq" rows refer to test sequences of statements, often formatted in a JUnit-like structure. They contain exactly what the "seq" row has, with

additional validation assertions, that are shown in 4.6. These assertions confirm that outputs exist and meet certain conditions. However, this validation is redundant because the dataset's output rows already capture the results of test executions.

```
org.junit.Assert.assertNotNull(arrayList1);  
org.junit.Assert.assertNotNull(arrayList2);  
org.junit.Assert.assertNotNull(arrayList3);
```

Figure 4.6.: JUnit Assertions in "exseq": "HumanEval_58_Common".

Ultimately, due to the reason that the rows where TYPE is "op", "seq", and "exseq" contained redundant data, they were removed from original data, leaving a total of 5722106 rows for further analysis.

4.3.3. Non-Functional Output Rows

The dataset now consists of rows representing input and output values. However, some rows contain non-meaningful outputs that do not contribute to the analysis. Specifically, rows where the column VALUE contains "_INSTANCE_" in the original data or "_NA_" in the oracle outputs were excluded.

In the original data, value "_INSTANCE_" indicates that an instance of the component under test was successfully created. However, this does not provide any functional behavior or meaningful output. Similarly, in the oracle outputs, "_NA_" reflects the absence of any functional output. After this filtering step, there were 4002576 rows remaining in the original data and 1719532 rows the oracle outputs. Specifically, the original data consists of 2283042 inputs and 1719534 outputs.

4.3.4. Irrelevant Columns

After filtering irrelevant rows, some irrelevant columns were excluded as well. The removed columns and their reasons are outlined below:

- EXECUTIONID: This column contains only two unique values: "821f389c-5b19-4da3-b7cf-2fe4610738b0" and "31a03823-2a33-4cce-8581-44761ba37e7a", which do not provide meaningful information for the current analysis.

- ACTIONID: This column always contains the value "execute".
- ARENAID: Similarly, ARENAID always contains the value "execute" after filtering out "jacoco" rows. Therefore, this column no longer serves any purpose in the analysis.
- ADAPTERID: This column identifies the adapter used during testing. However, its value is always "0" when VARIANTID is "original" and "oracle" when VARIANTID is "oracle". As this relationship is already captured by VARIANTID, the column was considered unnecessary.
- LASTMODIFIED: This column provides metadata about when each row was last updated. As this information does not contribute to understanding functional outputs or equivalence, it was excluded.
- EXECUTIONTIME: This column records the time taken for each test execution. While this may be relevant in dynamic analysis or performance evaluation, it is outside the scope of this research.

This refinement leaves the dataset with 10 columns, namely: ABSTRACTIONID, SHEETID, SYSTEMID, VARIANTID, X, Y, TYPE, VALUE, RAWVALUE, and VALUETYPE.

4.3.5. Data Type

It was observed that most columns of the SRM_full dataset were identified as strings ("object" data type), with the exception of the columns X and Y, which were correctly interpreted as "float64". This behavior could be caused by the export-import process, where metadata or the presence of mixed or null values might cause columns to default to strings. Since this research does not involve mathematical operations on the dataset, leaving the data in string format is sufficient for the intended analysis. Converting these columns to numerical or categorical types was considered unnecessary, as the dataset primarily serves as a basis for response type classification rather than quantitative computation. Additionally, the VALUETYPE column explicitly provides information about the data types of inputs and outputs, ensuring flexibility for any potential future analysis that may require type-specific processing. Therefore, to maintain the dataset's structure and avoid unintended modifications, the data was retained in its current string format.

Furthermore, data types (i.e., column VALUETYPE) of input and output rows were analyzed. The output rows exhibited consistent and expected behavior, with no anomalies in their VALUETYPE or VALUE columns. However, some input rows had VALUETYPE values starting with "HumanEval*", which seemed unusual for input data. To investigate further, these rows were analyzed, and it was found that they consistently had an input value of "{}", representing an empty dictionary. Many of these VALUETYPE entries corresponded to functional abstractions and ended with ".Problem", which is typically associated with abstraction identifiers rather than input types.

To determine whether other VALUETYPE entries were associated with "{}" as an input, the dataset was further examined. This revealed that there were some rows with VALUETYPE set to "java.util.HashMap", indicating that in these cases, "{}" was successfully interpreted as an empty map in Java.

Given this distinction, the outputs for rows with "{}" as input were analyzed, grouped by whether the VALUETYPE was "java.util.HashMap" or a functional abstraction name. The results showed that for rows where VALUETYPE was "java.util.HashMap", the outputs included: 1151 occurrences of "false", 432 occurrences of "true", 9 occurrences of "_EXCEPTION_", 2 occurrences of "B": "Blue", "W": "White", "G": "Green", "L": "Lavender", 1 occurrence of "B": "Black", "W": "White", "G": "Green", "O": "Orange".

For rows where VALUETYPE corresponded to a functional abstraction name (i.e., starting with "HumanEval*"), the outputs included: 35082 occurrences of "_EXCEPTION_", 1256 occurrences of "false" (all associated with "HumanEval_138_is_equal_to_sum_even"), 1 occurrence of "_NOT_EXECUTED_" (associated with "HumanEval_100_make_a_pile").

The presence of "false" and "true" outputs for "java.util.HashMap" entries suggests that some functional abstractions successfully processed "{}" as an input, producing valid outputs. Conversely, the rows with "HumanEval" as VALUETYPE showed a majority of exceptions, indicating a failure to handle "{}" effectively in those cases.

As a result, rows with "{}" as input were kept in the dataset because they provide useful insights into how the code candidates for functional abstractions handles empty inputs. The presence of "java.util.HashMap" as a VALUETYPE indicates

that "}" was processed differently by code candidates during testing, making these rows valuable for understanding this behavior in future analysis.

4.3.6. Handling Missing Values and Duplicates

In data analysis, preprocessing steps, such as data standardization, normalization, or scaling, are commonly applied in data analysis, however they were not performed. This decision is based on the nature of the dataset, because, transforming could distort the original behavior and functionality captured in the data. Instead, the focus was on identifying and addressing issues such as missing values and duplicates, while preserving the integrity of the original dataset.

In oracle outputs, no missing values were identified, confirming that all rows are complete and ready for analysis. In the original data, 43 rows were initially flagged as having missing values in the columns RAWVALUE and VALUETYPE. However, these rows correspond to rows where columns VALUE is `_NOT_EXECUTED_`, indicating that the associated tests were not executed. Therefore, the absence of RAWVALUE and VALUETYPE in such cases is expected and does not represent incomplete or erroneous data. Furthermore, these rows were retained in the dataset to preserve the full scope of test scenarios.

As for duplicates, it was revealed that there were no duplicate rows in either original data or oracle outputs, ensuring that all rows are unique and represent distinct test cases.

As a result of the preprocessing steps, the original dataset now contains 4002576 rows and 10 columns. Figure 4.7 illustrates the comparison of the total number of rows, unique SHEETIDs, and unique SYSTEMIDs between the HumanEval and MBPP benchmarks after the preprocessing steps. This comparison highlights the differences in the scale and structure of these two benchmarks, offering insights into the dataset's composition and its suitability for further analysis.

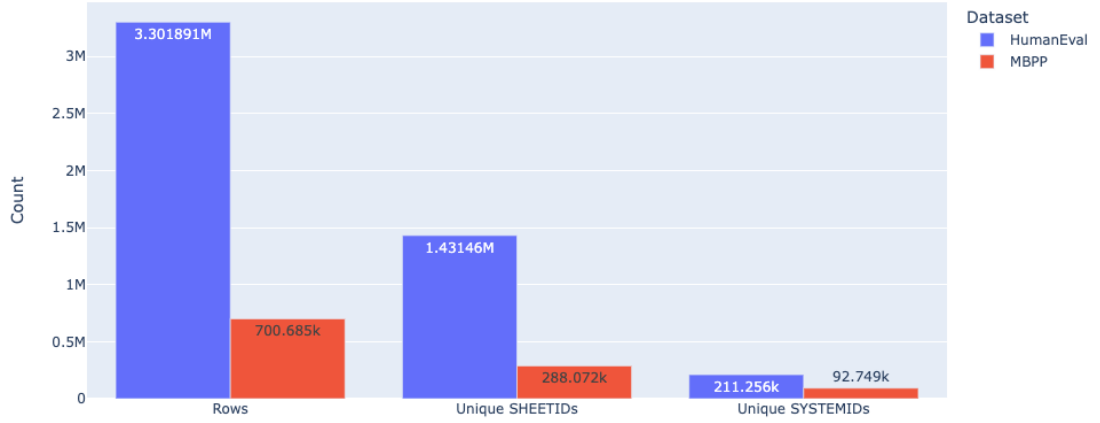


Figure 4.7.: Comparison of rows, Sheet IDs, and System IDs between HumanEval and MBPP after preprocessing steps.

4.3.7. Deriving Matched and Non-matching Data

As the last step of data preparation, the original outputs were compared to the oracle outputs based on their VALUE column. This process started by temporarily excluding inputs from the original data, as the oracle outputs do not contain such data. This comparison resulted in two datasets: a matching dataset, containing 903524 rows (52.54%), where the outputs matched the oracle outputs, and a non-matching dataset, containing 816010 rows (47.46%), where the outputs did not match. Notably, almost half of the outputs did not align with the oracle outputs, highlighting the extent of discrepancies between generated and expected results.

Both datasets now contain 19 columns, including ABSTRACTIONID, SHEETID, and eight columns each for the original and oracle outputs: SYSTEMID, VARIANTID, X, Y, TYPE, VALUE, RAWVALUE, and VALUETYPE, differentiated by suffixes _1 and _5, which refer to Variation 1 (original outputs) and Variation 5 (oracle outputs), respectively. The last column, "output_match", indicates "True" for matched outputs and "False" for non-matching outputs.

In addition, to analyze error-specific mismatches, the non-matching dataset was further classified based on the RAWVALUE column. Furthermore, the remaining rows that did not contain any errors, i.e., the "cleaned non-matching" dataset, were derived by removing the above error types from the non-matching dataset,

leaving 667613 rows for subsequent exploration. Figure 4.8 illustrates the distribution of the output rows in each dataset.

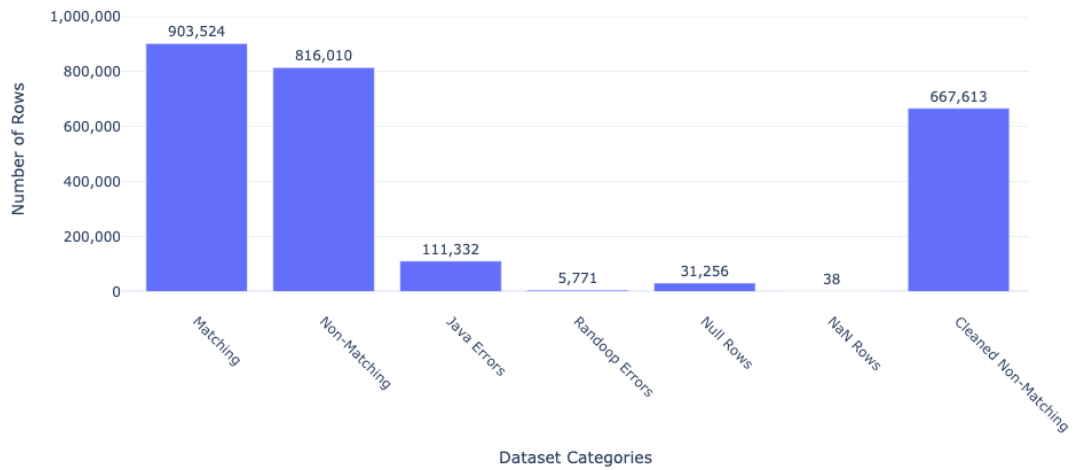


Figure 4.8.: Output row distribution across datasets.

Finally, all these seven datasets were merged with their corresponding inputs from the original dataset to incorporate the excluded inputs. This ensured that each dataset contained both the inputs and their respective outputs, enabling further input-output analysis.

5. Matched Data Analysis

This chapter focuses on analyzing matching data that was derived in Section 4.3.7. First, Section 5.1 identifies abstraction clusters where functional abstractions share identical input-output pairs. Then, Section 5.2 examines prompts and test cases of each functional abstraction within these clusters to determine whether they represent genuine functional relationship or mere overlaps.

5.1. Abstraction Clusters

The matching dataset consists of 903524 rows, representing cases where the generated outputs align with the oracle outputs. Within this dataset, there are 157 unique ABSTRACTIONIDs from HumanEval and 276 unique ABSTRACTIONIDs from MBPP, covering a wide range of functional tasks. In addition to abstraction diversity, the dataset also includes variations in SHEETIDs and SYSTEMIDs across the two benchmarks, as shown in Figure 5.1.

To enable an input-output analysis for each Sheet ID, the first step involved identifying input-output pairs for the dataset. Since each Sheet ID can have multiple inputs and outputs, the data was first grouped into input groups and output groups using Sheet IDs and Abstraction IDs as key identifiers. These grouped inputs and outputs were then merged to derive the input-output pairs. The resulting refined dataset included only four columns: ABSTRACTIONID, SHEETID, input_values (a list of all inputs for the corresponding SHEETID), output_values (a list of all outputs for the corresponding SHEETID). This structure facilitated a detailed input-output analysis, providing insights into the relationships and behaviors within the matched data.

Next, the Abstraction IDs that had the same input-output pairs were clustered together. For each cluster, all Sheet IDs associated with the same input-output pairs were grouped into a column "sheetid_list" and all corresponding Abstraction IDs were grouped into a column "abstractionid_list".

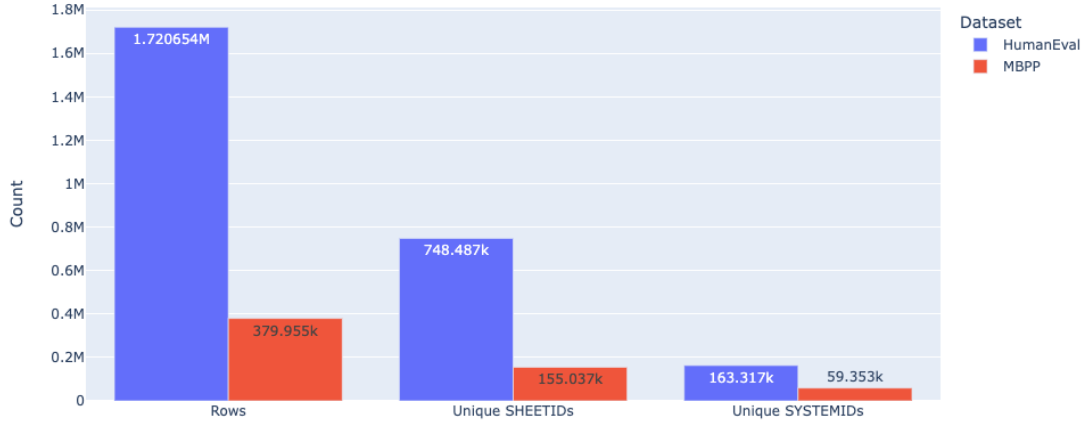


Figure 5.1.: Comparison of rows, Sheet IDs, and System IDs between HumanEval and MBPP benchmarks in matched dataset.

Clusters were only formed if the "abstractionid_list" contained more than one Abstraction ID, as clusters with a single Abstraction ID indicated no shared behavior across functional abstractions. This process resulted in 89 clusters, where multiple Abstraction IDs shared identical input-output pairs. However, it was observed that some clusters contained the same Abstraction IDs but had different input-output pairs. An example of how the new dataset looked and 4 clusters with same ABSTRACTIONIDs: "mbpp_127_multiply_int" and "mbpp_458_rectangle_area", are shown in Figure 5.2.

input_values	output_values	sheetid_list	abstractionid_list
('10', '20')	('200',)	['Problem_0_00c962d9-e210-4d2c-b226-978', 'Problem_0_3d2238aa-2594-4c05-a667-a632']	['mbpp_127_multiply_int', 'mbpp_458_rectangle_area']
('10', '5')	('50',)	['Problem_1_00c962d9-e210-4d2c-b226-978', 'e22c-4f31-89cd-53342b3be602', 'Problem_1_']	['mbpp_127_multiply_int', 'mbpp_458_rectangle_area']
('20', '10')	('200',)	['Problem_0_01d26c0f-1b0b-4b92-a0fb-2227', 'dd2a-4bbd-937e-7d993b11f050', 'Problem_0_']	['mbpp_127_multiply_int', 'mbpp_458_rectangle_area']
('5', '10')	('50',)	['Problem_1_02eca0b9-45e4-45e2-886a-fec5', 'Problem_1_2e920738-9441-4d9f-b33e-fd55t']	['mbpp_127_multiply_int', 'mbpp_458_rectangle_area']

Figure 5.2.: Clusters of "mbpp_127_multiply_int" and "mbpp_458_rectangle_area" with different input-output pairs.

When grouping by unique combinations of Abstraction IDs (ignoring different input-output pairs), there were 69 unique abstraction clusters. However, the clusters with duplicate Abstraction IDs were retained because each input-output pair, along with its associated Sheet ID, could reveal important details about the behavior and testing of these abstractions. As further analysis involves check-

ing the assertions provided in the descriptions of HumanEval and MBPP tasks, these assertions offer valuable context about the functional expectations for each abstraction and help explain why certain abstractions share similar input-output pairs across clusters.

5.2. Relationship Analysis

This section analyzes the prompts and test cases of each functional abstraction of each cluster to understand why specific Abstraction IDs were grouped together. The purpose is to determine whether the input-output pairs observed in the clusters align with the descriptions, test cases, or assertions provided in the HumanEval and MBPP datasets. Furthermore, the analysis aims to establish whether the abstractions in a cluster were simply overlapping (i.e., sharing similar input-output pairs) or if there were deeper functional or behavioral relationships between them.

To achieve this, the Java-translated versions of the HumanEval and MBPP benchmark datasets, provided by the MultiPL-E is used [18]. Each dataset contains several columns, including "name", "language", "prompt", "doc-tests", "original", "prompt_terminology", "tests", and "stop_tokens". For this study, only three columns were used: "name", which uniquely identifies each task and its origin (e.g., HumanEval_31_is_prime or mbpp_72_dif_Square); "prompt", which provides the problem description or task statement; and "tests", which include the test cases used to validate the solutions. The column "language" was excluded since it is set to Java, and other columns, such as "doctests", "original", "prompt_terminology", and "stop_tokens", were irrelevant to this analysis.

Categories and Criteria

69 unique clusters that were extracted in Section 5.1 were further analyzed to determine the relationships between their abstractions. This analysis was guided by an examination of prompts, and test cases associated with each abstraction. Three key criteria were applied to classify these relationships: behavioral equivalence, functional similarity, and contextual relationship. Based on these criteria, the clusters were categorized as follows:

- Behaviorally equivalent clusters (1 Cluster): Abstractions that exhibit iden-

tical input-output behavior for the test cases. These clusters are also inherently functionally similar and contextually related, as behavioral equivalence implies shared functionality and contextual alignment.

- Partially behaviorally equivalent clusters (2 Clusters): Abstractions that share contextual relationships, functional similarity, and partial behavioral equivalence for specific test cases, which means that it produces overlaps for some inputs but diverges for others.
- Functionally similar and contextually related clusters (20 Clusters): Abstractions that are both contextually related and exhibit functional similarity.
- Contextually related clusters (4 Clusters): Abstractions that share a similar context or domain but do not exhibit functional similarity or behavioral equivalence.
- Functionally similar clusters (6 Clusters): Abstractions that perform similar tasks with overlapping logic but are not identical.
- Identical clusters (1 Cluster): Abstractions that are functionally identical, with the same purpose, implementation, and behavior.
- Unrelated clusters (35 Clusters): Abstractions that are not behaviorally equivalent, functionally similar, or contextually related.

Each category highlights a distinct type of relationship, reflecting the diversity of interactions between abstractions. A detailed breakdown of these categorized abstraction clusters is provided in Appendix A1.

5.2.1. Behaviorally Equivalent Clusters

The behaviorally equivalent cluster includes the functional abstractions "mbpp_458_rectangle_area" and "mbpp_127_multiply_int". These abstractions were grouped together due to their identical input-output behavior for specific test cases. The input-output pairs for this cluster are shown in the Figure 4.1.

Although, both of these return multiplication of given two integers, the implementations of these abstractions differed significantly. The "mbpp_458_rectangle_area" function calculates the area of a rectangle by directly multiplying its

length and breadth, using a straightforward approach to return the product of two numbers (see Listing 5.1). In contrast "mbpp_127_multiply_int" employs a recursive approach to perform multiplication without using the "*" operator, relying instead on repeated addition and conditional logic (see Listing 5.2.)

These abstractions exhibit behavioral equivalence for overlapping test cases due to their shared reliance on arithmetic operations. However, they are not functionally equivalent as their purposes differ: one abstraction calculates a geometric property while the other provides a general solution for multiplication. In addition, they are functionally similar due to their shared arithmetic logic and exhibit a contextual relationship, as both operate within the domain of positive integer inputs.

```
1 class Problem {  
2     public static long rectangleArea(long l, long b) {  
3         return l * b;  
4     }  
5  
6     public static void main(String[] args) {  
7         assert(rectangleArea(10L, 20L) == 200L);  
8         assert(rectangleArea(4L, 2L) == 8L);  
9     }  
10 }
```

Listing 5.1: Java implementation of "mbpp_458_rectangle_area".

```
1 class Problem {  
2     public static long multiplyInt(long x, long y) {  
3         if (y < 0) return -multiplyInt(x, -y);  
4         else if (y == 0) return 0;  
5         else if (y == 1) return x;  
6         return x + multiplyInt(x, y - 1);  
7     }  
8  
9     public static void main(String[] args) {  
10        assert(multiplyInt(10L, 20L) == 200L);  
11        assert(multiplyInt(4L, 8L) == 32L);  
12    }  
13 }
```

Listing 5.2: Java implementation of "mbpp_127_multiply_int".

5.2.2. Partially Behaviorally Equivalent Clusters

Two clusters were identified as partially behaviorally equivalent, reflecting both shared logic and domain-specific similarities, while also highlighting behavioral differences under certain input conditions. Their input-output mappings are shown in Table 5.1.

Cluster	input_values	output_values
mbpp_736_left_insertion, mbpp_786_right_insertion	('3', '[1,2,4,5]')	('2',)
	('6', '[1,2,4,5]')	('4',)
	('7', '[1,2,4,5]')	('4',)
	('1,2,4,5]', '3')	('2',)
	('1,2,4,5]', '6')	('4',)
	('1,2,4,5]', '7')	('4',)
mbpp_790_even_position, mbpp_775_odd_position	('2,3]',)	('false',)
	('1,2,3]',)	('false',)

Table 5.1.: Partially Behaviorally Equivalent Clusters.

The first cluster consists of "mbpp_736_left_insertion" and "mbpp_786_right_insertion", which identify insertion points for a value in sorted lists. The former one inserts a value into a sorted list from the left side and locates its point, and the latter one focuses on the right side. For the given inputs in the tests, all input-output pairs matched. For example, with inputs "3, [1,2,4,5]" and "[1,2,4,5], 3", both abstractions return "2", correctly identifying the appropriate insertion point.

It is obvious that all the provided test inputs contain unique values in the list, ensuring that the outputs always align. However, when considering scenarios with duplicates, their behaviors might diverge. For instance, in the sorted list [1,2,4,5,5] with the value "5" (a hypothetical input), the abstraction "mbpp_736_left_insertion" would return "3", prioritizing the first occurrence of the duplicate, while "mbpp_786_right_insertion" would return "5", placing the value after the last occurrence of the duplicate.

It is also important to note that the prompts for both abstractions specify behavior for "sorted order" but make no explicit mention of handling duplicates. This could imply that the intended inputs are expected to contain only unique values. If this assumption holds true, the abstractions are behaviorally equivalent, as their outputs match perfectly for all unique-value inputs provided in the tests.

However, since no inputs involving duplicates were included in the data, this remains only a consideration based on the potential behavior of the abstractions under such conditions.

The second cluster includes two abstractions: "mbpp_790_even_position" and "mbpp_775_odd_position." These functions analyze parity constraints in lists i.e., targeting even and odd indices, respectively. For example, for inputs like [1,2,3], both functions correctly return "false", as the values violate the parity conditions for their respective indices. However, for inputs like [2,3], both functions incorrectly return "false", even though the input satisfies the parity constraints. Upon further investigation, it was observed that the oracle and original outputs for [2,3] are also incorrectly labeled as false, suggesting an error in the oracle. Furthermore, no explicit assertion for [2,3] exists in the original benchmark. Despite this anomaly, the abstractions share similar logic in identifying valid or invalid parity conditions at specified positions, while maintaining their focus on different index types.

Finally, even though both clusters show similar behavior in some cases, they highlight different priorities in how they are designed. The first cluster is about finding insertion points in a sorted list, with one abstraction prioritizing the leftmost position and the other focusing on the rightmost position. The second cluster focuses on positions in a list, with one abstraction checking even indices and the other checking odd ones. This shows that even when abstractions have the same overall goal, small differences in their design can lead to variations in how they work. Ultimately, these abstractions are partially behaviorally equivalent.

5.2.3. Functionally Similar and Contextually Related Clusters

There were found 20 abstraction clusters that are both contextually related and functionally similar. These clusters highlight abstractions that operate on related inputs, often using similar logic or approaches, while addressing tasks within the same domain. The clusters are further grouped into three categories based on their interconnections and relationships: highly connected, broadly connected and self-contained clusters.

Highly connected clusters

First two clusters include "mbpp_787_text_match_three", "mbpp_756_text_match_

zero_one", "mbpp_285_text_match_two_three", and "mbpp_434_text_match_one". These abstractions focus on tasks involving text pattern matching, specifically determining whether a string follows certain structural patterns involving the characters "a" and "b". The first cluster includes all of these abstractions and returns "false" for inputs "ac" and "dc". The second cluster includes the first three abstractions and for input "abbbba" returns "true".

These functions share a similar purpose, however they are not behaviorally equivalent as each checks for distinct patterns. For example:

- "mbpp_787_text_match_three" matches strings where "a" is followed by exactly three "b"-s;
- "mbpp_756_text_match_zero_one" matches strings where "a" is followed by one or more "b"-s;
- "mbpp_285_text_match_two_three" checks for two or three "b"-s following "a";
- "mbpp_434_text_match_one" matches strings where "a" is followed by one or more "b"-s, with slight implementation differences compared to "mbpp_756_text_match_zero_one".

These abstractions are not functionally equivalent, as their outcomes depend on distinct regular expression constraints. However, they are functionally similar as they all operate within the same domain of regular expression-based string pattern matching. Finally, their shared purpose of identifying patterns involving "a" and "b" places these functions in the same domain, making them contextually related.

Broadly Connected Clusters

Next, there are 8 clusters formed by various combinations of the following functional abstractions: "mbpp_751_check_min_heap", "HumanEval_126_is_sorted", "mbpp_472_check_Consecutive", "mbpp_281_all_unique", "HumanEval_109_move_one_ball", "mbpp_760_unique_Element", "mbpp_19_test_duplicate", and "HumanEval_57_monotonic". These abstractions share a contextual relationship and functional similarity as they all focus on analyzing sequence properties or structural features, such as checking for sorting, uniqueness, monotonicity, or specific constraints. Their connection is shown in Figure 5.3.

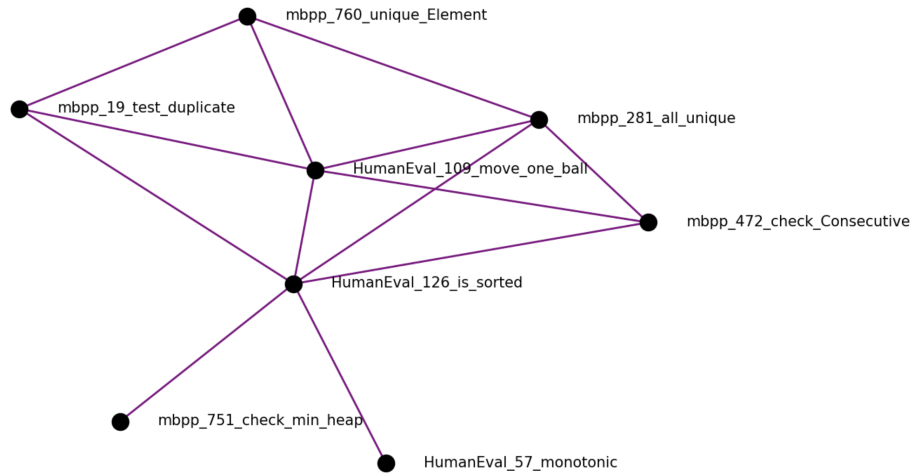


Figure 5.3.: Broadly connected clusters

Each cluster includes abstractions that address related tasks, often with overlapping goals but distinct implementations. For example, abstractions like "HumanEval_126_is_sorted" and "mbpp_751_check_min_heap" frequently appear together, focusing on verifying whether a sequence or structure adheres to certain ordering rules. Similarly, abstractions such as "mbpp_281_all_unique" and "mbpp_760_unique_Element" assess whether sequences contain duplicate elements, while "mbpp_19_test_duplicate" tackles the identification of duplicates directly. Functions like "HumanEval_57_monotonic" and "mbpp_472_check_Consecutive" examine whether sequences maintain consistent patterns, such as monotonic increases or consecutive elements.

Although these abstractions are not behaviorally equivalent, since their outputs depend on the specific properties being evaluated, they are functionally similar because they operate within the same domain of sequence and structural analysis. Their shared logic and purpose create a strong contextual connection, linking tasks like ordering, uniqueness verification, and pattern detection.

Self-contained Clusters

Finally, there are 10 clusters that are self-contained, meaning there is a relationship only within their respective groups. Each group addresses tasks within specific problem domains, such as text pattern matching, numerical evaluation, or sequence generation, while maintaining contextual and functional similarities inter-

nally. For instance, "mbpp_454_text_match_wordz" and "mbpp_643_text_match_wordz_middle" focus on string pattern matching. The first abstraction checks for the presence of the letter "z" in any part of the string, while the second adds the constraint of ensuring that "z" is not at the start or end. These differences highlight how even closely related abstractions can diverge based on subtle variations in task requirements.

Numerical analysis appears as a recurring theme across several clusters, including ["mbpp_435_last_Digit", "mbpp_577_last_Digit_Factorial"], ["HumanEval_24_largest_divisor", "mbpp_295_sum_div"], and ["HumanEval_142_sum_squares", "mbpp_798_sum"]. The former cluster focuses on identifying the last digit of numbers, with one abstraction applied to integers and the other to factorials, illustrating how shared computational principles can be adapted to different contexts. Similarly, the second explores divisor analysis, where one abstraction identifies the largest divisor smaller than the input, and the other calculates the sum of all divisors. The last cluster performs summation on lists of numbers with overlapping logic, where the first abstraction applies transformation before summing, and the other directly computes the total.

Other clusters, such as ["HumanEval_74_total_match", "HumanEval_58_common"] and ["HumanEval_29_filter_by_prefix", "HumanEval_7_filter_by_substring"] focus on comparisons and filtering tasks. The list comparison cluster highlights how two abstractions share the goal of evaluating similarities or differences in lists while differing in their outputs: one identifies common elements, while the other compares total characters to return the smaller list. Similarly, the filtering cluster targets specific conditions for string inclusion, either based on substrings or prefixes. These clusters show how abstractions in the same domain often approach problems with varied perspectives, emphasizing precision and specialization.

The sequence-based cluster, ["HumanEval_63_fibfib", "HumanEval_46_fib4"], involves generating "Fibonacci-like sequences" but applies distinct rules to determine each element. While both abstractions rely on similar computational logic, one sums the previous three numbers, and the other sums the last four, reflecting different definitions of sequence construction. A similar divergence is seen in the subset evaluation cluster, ["HumanEval_43_pairs_sum_to_zero", "HumanEval_40_triples_sum_to_zero"]. Both abstractions analyze lists for subsets summing to zero but differ in their subset sizes, one evaluating pairs and the

other triples. These clusters highlight how abstractions adapt common computational strategies to distinct problem specifications.

The cluster ["HumanEval_16_count_distinct_characters", "mbpp_172_count_occurrence", "HumanEval_98_count_upper", "HumanEval_23_strlen", "HumanEval_66_digitSum"] focuses on counting and measuring tasks within strings. While these abstractions may return similar results for specific inputs (e.g., an empty string), the equivalence is coincidental and does not reflect shared logic. Each abstraction addresses unique objectives, such as counting distinct characters, occurrences of a substring, or ASCII values of uppercase characters, creating contextual and functional similarities.

Finally, a cluster that with 13 functional abstractions "HumanEval_104_unique_digits", "HumanEval_22_filter_integers", "HumanEval_42_incr_list", "HumanEval_9_rolling_max", "HumanEval_145_order_by_points", "HumanEval_88_sort_array", "HumanEval_116_sort_array", "HumanEval_26_remove_duplicates", "HumanEval_30_get_positive", "HumanEval_68_pluck", "HumanEval_70_strange_sort_list", "HumanEval_105_by_length", "HumanEval_149_sorted_list_sum" has a broad range of tasks related to filtering, sorting, and transforming lists. While behaviorally equivalent outputs may occasionally occur (e.g., for empty inputs), these abstractions are fundamentally distinct in their logic and purpose. For example, some abstractions focus on identifying unique digits, removing duplicates, or sorting lists based on custom rules like binary representations or sum of digits, highlighting the versatility and contextual relationship among these tasks.

5.2.4. Functionally Similar Clusters

Six clusters of functionally similar abstractions were identified. These clusters demonstrate varying degrees of similarity in logic and operations, despite differences in purpose and behavior. The identified clusters, their input-output pairs, and analysis are presented in Table 5.2.

These clusters share several key characteristics. Functional similarity often arises from shared computational techniques. For instance, "mbpp_797_sum_in_range" and "HumanEval_53_add" rely on addition, while "mbpp_296_get_Inv_Count" and "HumanEval_73_smallest_change" involve counting properties of arrays. Similarly, "mbpp_68_is_Monotonic" and "mbpp_760_unique_Element" evaluate global

Cluster	Input-Output Pairs	Insights and Observations
mbpp_797_sum_in_range, HumanEval_53_add	('5', '7') → ('12',)	Both abstractions return the same result for this input, but their objectives differ: one sums odd numbers in a range, while the other performs general-purpose addition.
mbpp_296_get_Inv_Count, HumanEval_73_smallest_change	(new ArrayList<> ([1, 20, 6, 4, 5])) → 5, (new ArrayList<> ([1, 2, 3, 5, 4, 7, 9, 6])) → 4	Both count properties in an array but have distinct goals: inversion counting vs. palindromic transformation.
mbpp_68_is_Monotonic, mbpp_760_unique_Element	[1, 2] → "false"	Shared boolean output for this input, but one checks monotonicity while the other evaluates uniqueness.
mbpp_790_even_position, HumanEval_126_is_sorted	[3, 2, 1] → "false"	Both return "false," yet one checks if even indices contain even numbers, while the other evaluates sorting.
mbpp_68_is_Monotonic, mbpp_775_odd_position	[1, 3, 2] → "false"	Both analyze structural properties of arrays: one for monotonicity, the other ensuring odd indices contain odd numbers.
mbpp_68_is_Monotonic, mbpp_775_odd_position, mbpp_790_even_position	[1, 2, 3] → "false"	Expands Cluster 5 by adding "even position," forming a subcluster focused on position-based checks and monotonicity evaluation.

Table 5.2.: Functionally similar clusters.

properties of arrays, returning boolean results based on specific evaluations. Another commonality lies in the focus on structural or numerical properties of data, particularly lists or arrays. Many abstractions, such as "mbpp_790_even_position", which checks whether even indices contain even numbers, and "HumanEval_126_is_sorted", which evaluates the order of elements in a list, highlight this recurring theme. However, despite functional similarity, behavioral equivalence is rarely observed. Matching outputs for specific inputs are often coincidental rather than indicative of shared logic. For example, "mbpp_68_is_Monotonic" and "mbpp_760_unique_Element" both return false for [1, 2], but their logic diverges significantly for other inputs.

These clusters also reveal notable contrasts between specialized and generalized logic. For example, "mbpp_797_sum_in_range" focuses on summing odd numbers within a range, while "HumanEval_53_add" is a general-purpose addition function. This contrast demonstrates how similar computational operations can be adapted to solve distinct problems.

Furthermore, the contextual relationships between abstractions within the same cluster are weak compared to the clusters that were categorized as both functionally similar and contextually related. For instance, a cluster that consists "mbpp_296_get_Inv_Count" and "HumanEval_73_smallest_change" exhibit weak contextual ties as they address unrelated goals (array inversions versus palindromic transformations). Similarly, "mbpp_790_even_position" and "HumanEval_

"126_is_sorted" show only weak contextual overlap, as their tasks are distinct (i.e., validating list properties versus ensuring even index constraints). As a result, they were categorized as functionally similar but not contextually related.

A notable observation is how abstractions with similar functionality can serve entirely different purposes and exhibit distinct behaviors. For instance, "mbpp_68_is_Monotonic" and "mbpp_775_odd_position" analyze structural properties of arrays but focus on distinct criteria. Furthermore, it indicates the complexities of grouping abstractions solely based on functional similarity and emphasize the importance of considering their unique objectives and contexts.

5.2.5. Contextually Related Clusters

Contextually related clusters consist of functional abstractions that share a broader contextual relationship, such as operating on similar input types or belonging to the same problem domain, but do not exhibit functional similarity or behavioral equivalence. These clusters highlight abstractions that interact with related data types or processes without overlapping in logic or behavior. Below, the details of the four contextually related clusters are discussed.

The first cluster includes the abstractions "HumanEval_108_count_nums" and "HumanEval_69_search", both of which process numerical inputs and produce numerical outputs. For the input [1], both abstractions yield the output "1". Despite this similarity, their purposes and logic are fundamentally distinct. The "count_nums" function focuses on digit-level arithmetic, while the "search" function provides a comparison between two values. Consequently, the functions are neither behaviorally nor functionally equivalent. However, they share a contextual relationship as they both interact with numerical inputs and outputs, placing them within the broader domain of numerical operations.

The second cluster consists of three abstractions, including: "HumanEval_14_all_prefixes", "HumanEval_17_parse_music", and "HumanEval_101_words_string". These abstractions share a focus on string processing but perform entirely different tasks. For example, for the input "", all three functions return "". The "all_prefixes" function generates all prefixes of a given string, "parse_music" converts musical notes into beats, and "words_string" splits a string into individual words. These distinct purposes mean the abstractions are neither behaviorally

nor functionally equivalent. However, they share a contextual relationship as they belong to the broader category of string manipulation, demonstrating how abstractions can be related through their input types and high-level domain focus despite diverging logic and applications.

The third cluster in this category consists of "HumanEval_83_starts_one_ends", "mbpp_592_sum_of_product", "HumanEval_55_fib", "HumanEval_60_sum_to_n", and "HumanEval_139_special_factorial". These functions all involve mathematical computations and often rely on iterative or recursive processes. For the input "1", the output across all functions is "1". However, the distinct logic and purposes of each function preclude behavioral or functional equivalence. While they all involve numerical computations, the differences in their specific implementations and objectives weaken the functional similarity between them. Nevertheless, they share a contextual relationship, as they operate on integers and return numerical results, frequently employing mathematical operations such as sums, products, or combinatorics.

The fourth cluster consists of five abstractions: "HumanEval_27_flip_case", "HumanEval_10_make_palindrome", "HumanEval_51_remove_vowels", "HumanEval_86_anti_shuffle", and "HumanEval_19_sort_numbers". For the input "", all functions return the output "". These abstractions are united by their focus on string processing, where the input is a string, and the output is a transformed version of the string based on specific logic. However, their purposes and logic diverge significantly. Therefore, these clusters have a contextual relationship.

Looking at the clusters as a whole, a clear pattern emerges: these functions take in specific inputs, like numbers or strings, and analyze or process them to produce something meaningful. For instance, one function might focus on counting numbers, while another compares them, yet both are still grounded in working with numerical data. Similarly, within string-focused clusters, functions transform or reorganize strings based on unique objectives. This diversity in purpose and logic underscores why these abstractions are only contextually related, even as they share similar input types or high-level domains.

5.2.6. Identical Abstractions

During this analysis, two functional abstractions, "mbpp_591_swap_List" and "mbpp_625_swap_List", were found to be identical. The input-output pairs for these abstractions reinforce their identical nature: for the inputs [1, 2, 3] and [4, 5, 6], both abstractions return [3, 2, 1] and [6, 5, 4], respectively.

The implementations of these abstractions further confirm their equivalence. Both abstractions implement the task of interchanging the first and last elements in an array list. The logic to perform the task is nearly identical in both cases, with minor variations in the test cases and formatting.

Interestingly, these abstractions were observed to belong to different data splits within the MBPP dataset. The "mbpp_591_swap_List" abstraction is part of the validation set, while "mbpp_625_swap_List" is included in the training set. This observation suggests that the inclusion of identical abstractions in different splits may have been intentional. By including equivalent tasks in both the training and validation datasets, the benchmark likely aims to validate whether the model can correctly solve tasks during testing that it encountered during training.

5.2.7. Unrelated Clusters

As for unrelated clusters, there were identified a total of 35 clusters. Among these, 7 clusters were deemed completely unrelated, showing no behavioral equivalence, functional equivalence, functional similarity, or contextual relationship. The remaining 28 clusters exhibited very weak contextual relationship, such as working with similar data types or loosely related problem domains, but were classified as unrelated due to their fundamentally different purposes and logic.

Completely Unrelated Clusters

The 7 completely unrelated clusters consist of abstractions serving entirely different purposes, with no shared logic, behavior, or operational goals. For example, the cluster ["HumanEval_162_string_to_md5", "HumanEval_111_histogram"] produces "{ }" for an input "", representing an empty MD5 hash and an empty frequency, respectively. Therefore, these tasks are addressing entirely different domains with no overlap in purpose or functionality.

Another example is the cluster ["HumanEval_73_smallest_change", "HumanEval_108_count_nums"], with input [0,1] and output "1" for both abstractions. "HumanEval_73_smallest_change" determines the minimum number of changes required to make an array palindromic, while "HumanEval_108_count_nums" counts the elements in the array where the sum of digits exceeds zero. Despite the shared input-output pair, the matching result is purely coincidental and reflects no functional or contextual connection.

The cluster ["mbpp_762_check_monthnumber_number", "mbpp_406_find_Parity"] uses the input 12 and produces the output "false". Here, "mbpp_762_check_monthnumber_number" validates whether the input represents a valid month, while "mbpp_406_find_Parity" determines the binary parity of the input number. These functions target entirely different problem domains—calendar logic versus numerical parity—and have no shared logic or goals.

In ["mbpp_790_even_position", "HumanEval_43_pairs_sum_to_zero"], the input [3] and output "false" are coincidental. "mbpp_790_even_position" checks whether the value at a specific index in an array is even, while "HumanEval_43_pairs_sum_to_zero" identifies pairs of elements that sum to zero. These tasks involve unrelated goals and have no meaningful connection in purpose or behavior.

The cluster ["mbpp_802_count_rotation", "mbpp_62_smallest_num", "mbpp_166_find_even_pair"] processes the input [1,2,3] and outputs "1". Despite this common output, the tasks differ significantly: "mbpp_802_count_rotation" counts rotations in an array, "mbpp_62_smallest_num" identifies the smallest number in an array, and "mbpp_166_find_even_pair" finds even-pair combinations. These abstractions address distinct domains, with no shared operational logic.

Similarly, the cluster ["HumanEval_75_is_multiply_prime", "mbpp_803_is_perfect_square", "mbpp_72_diff_Square", "mbpp_406_find_Parity"] processes the input 10 and outputs "false". Each function evaluates distinct mathematical concepts: 1) whether a number is multiply prime, 2) a perfect square, 3) the difference between squares, or 4) binary parity. The shared input-output pair is purely coincidental and not indicative of any shared functionality or context.

Finally, in ["mbpp_164_are_equivalent", "mbpp_6_differ_At_One_Bit_Pos"], the input (2,4) and output "false" are unrelated. "mbpp_164_are_equivalent" evaluates equivalence under certain criteria, while "mbpp_6_differ_At_One_Bit_Pos" checks

whether two numbers differ at exactly one bit position. These functions operate in distinct problem domains with no shared logic or contextual overlap.

Unrelated Clusters with Weak Contextual Relationship

The remaining 28 clusters displayed very weak contextual relationships, often stemming from shared data types or superficial overlaps in purpose. Since it is too extensive to detail all 28 clusters, a few examples are discussed to illustrate the nature of their weak relationships.

For instance, in the cluster ["mbpp_804.is_product_even", "mbpp_281_all_unique"], both functions analyze lists but target fundamentally different characteristics. One checks whether the product of elements in the list is even, while the other determines whether all elements in the list are unique. Despite working on lists, their goals and logic are entirely distinct, and the relationship is purely superficial.

Similarly, the cluster ["mbpp_404_minimum", "HumanEval_97_multiply"] involves numerical computations, but the purposes diverge. "mbpp_404_minimum" identifies the minimum value between two numbers, while "HumanEval_97_multiply" performs a multiplication operation. The shared domain of numerical computations does not translate into meaningful functional or behavioral equivalence.

In ["mbpp_741_all_Characters_Same", "mbpp_113_check_integer"], both abstractions operate on strings but serve entirely different purposes. The first abstraction checks whether all characters in a string are identical, in contrast the second validates whether a string represents an integer. The weak contextual relationship stems from their shared input type (strings), but their goals and implementations are unrelated.

Next, in cluster ["mbpp_455_check_monthnumb_number", "mbpp_72_dif_Square", "HumanEval_31_is_prime"]. Although all three functions evaluate numerical properties, they operate in distinct domains. For example, "mbpp_455_check_monthnumb_number" deals with calendar logic, "mbpp_72_dif_Square" calculates differences between squares, and "HumanEval_31_is_prime" tests primality. These tasks are fundamentally different, and the contextual overlap is too weak to establish any meaningful connection.

Finally, the cluster ["mbpp_404_minimum", "mbpp_558_digit_distance_nums"] highlights how functions with similar inputs can still be unrelated. Both abstractions accept two numerical inputs and produce a numerical output. However,

"mbpp_404_minimum" compares the inputs to find the smallest value, while "mbpp_558_digit_distance_nums" performs digit-level arithmetic to compute a "digit distance." The weak contextual overlap of working with numerical inputs fails to establish functional or behavioral similarity.

These examples represent the general nature of the weak contextual relationships in these clusters, but the remaining clusters follow a similar pattern of superficial overlaps without substantive connections.

6. Non-matching Data Analysis

This chapter aims to provide a clearer understanding of the variations in output behavior and the underlying reasons behind them by analyzing the non-matching dataset. The non-matching dataset was derived in Section 4.3, and was further categorized into subsets based on the types of errors encountered. These subsets include: Java Exceptions, Randoop Timeouts, Null Rows, NaNs. On top of that, a cleaned non-matching dataset (i.e., valid non-matching outputs) was derived by removing rows associated with these errors. In the following subsections, each type of error/exception will be analyzed, exploring its causes, patterns, and their potential impact.

6.1. Java Exceptions

The Java exception dataset contains 170703 input rows, with 111332 corresponding output rows (equivalent to the number of unique Sheet IDs), distributed across 349 unique Abstraction IDs, and 24405 unique System IDs. The dataset includes 29 distinct java exception types, highlighting a broad spectrum of challenges encountered during the execution. Figure 6.1 shows the java exception rows, classified according to their frequency, into five categories.

The number of abstractions affected by each java exception is shown in Figure 6.2. As we can see, some java exceptions impacted a much larger number of functional abstractions. For example, "java.lang.IndexOutOfBoundsException" occurred in 180 functional abstractions, making it the most prevalent error. This was followed by "java.lang.StackOverflowError", which occurred in 143 abstractions, and "java.lang.ArrayIndexOutOfBoundsException", with occurrences in 126 abstractions. In contrast some java exceptions occurred in 18 or fewer functional abstractions, including "java.util.EmptyStackException", "java.lang.ArrayStoreException", "java.lang.Error", "java.util.IllegalFormatConversionException", "java.lang.IllegalStateException", and "java.lang.NegativeArraySizeException".

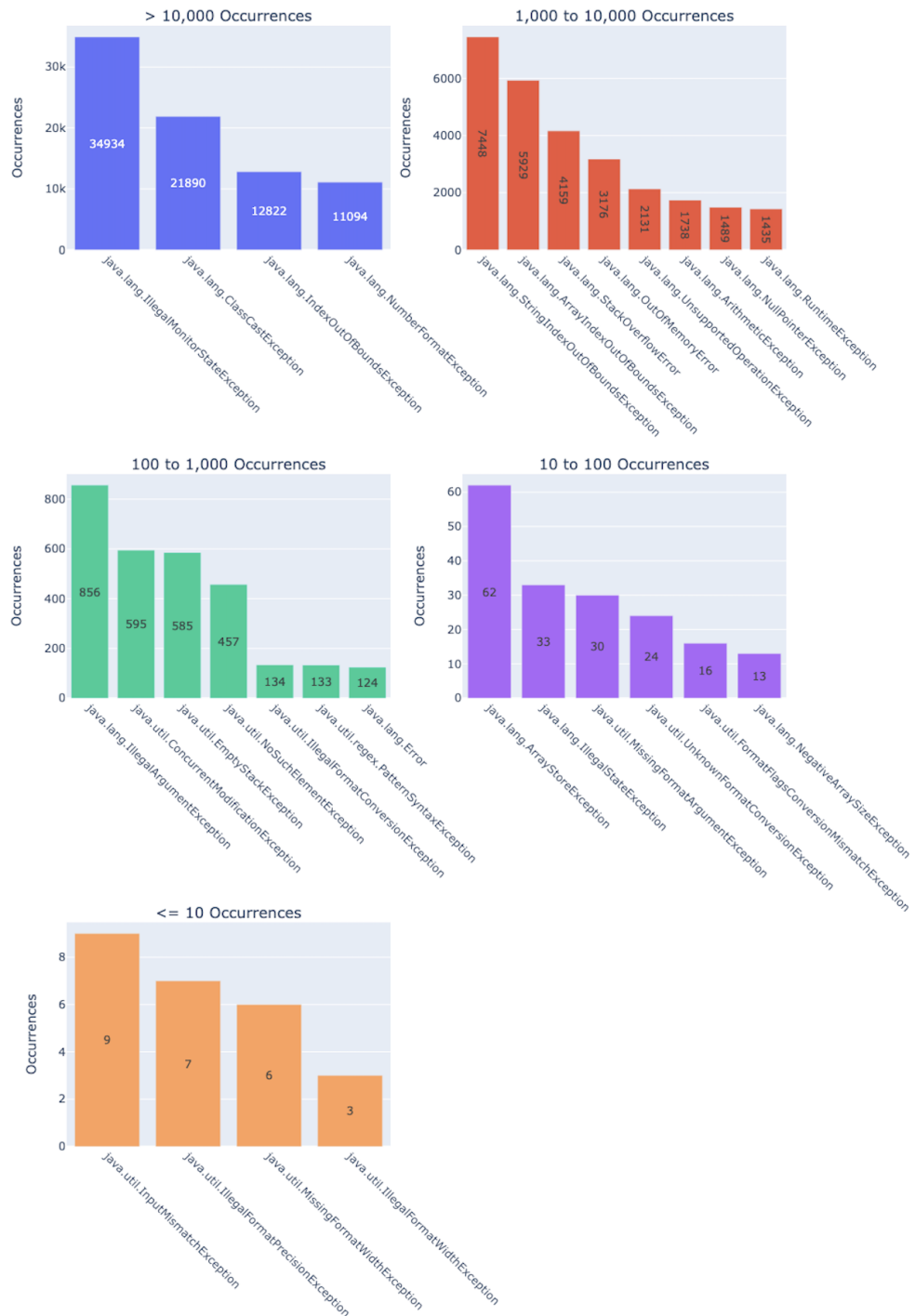


Figure 6.1.: Categorization of java exceptions by frequency.

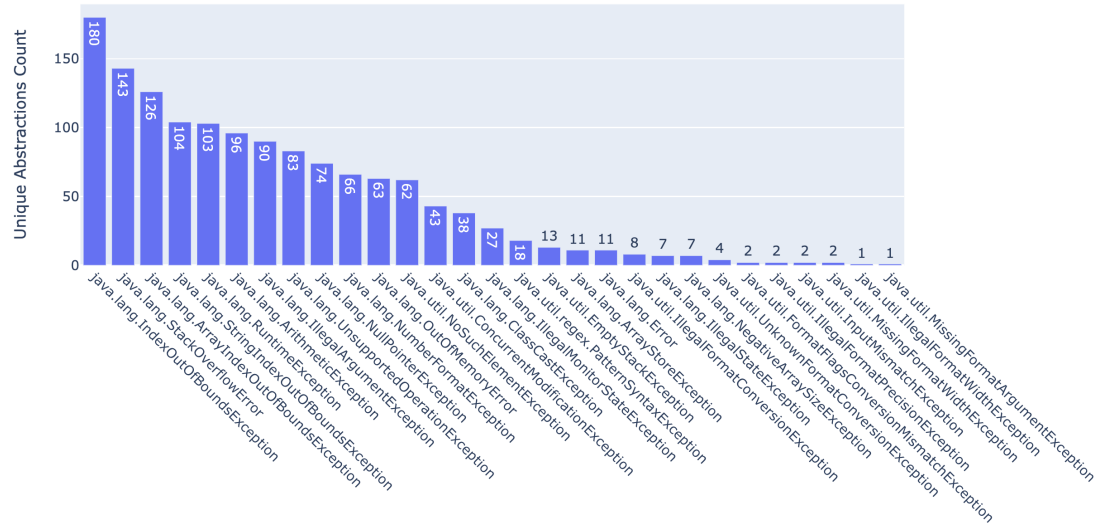


Figure 6.2.: Number of abstractions affected by each java exception.

Since this focuses more on the analysis of java exceptions rather than the abstractions themselves, the emphasis is on understanding why specific java exceptions occurred across various System IDs, rather than investigating how these exceptions relate to particular abstractions. The distribution of java exceptions among unique System IDs is shown in Figure 6.3.

The distribution varies significantly, with some exceptions impacting a large number of System IDs, while others are relatively rare. Among the most common exceptions, "java.lang.IllegalMonitorStateException" stands out, affecting 5085 unique System IDs. It is followed by "java.lang.ClassCastException", which affects 4794 System IDs, and "java.lang.IndexOutOfBoundsException", which affects 4478 System IDs. Other frequently occurring exceptions include "java.lang.StringIndexOutOfBoundsException" (2759 System IDs) and "java.lang.NumberFormatException" (2512 System IDs).

On the other hand, several java exceptions occur much less frequently, affecting only a few System IDs. For example, "java.util.IllegalFormatWidthException" is found in just 1 System IDs, while "java.util.InputMismatchException", "java.util.IllegalFormatPrecisionException", and "java.util.MissingFormatWidthException" each occur in only 2 System IDs. Slightly more prevalent exceptions, such as "java.util.UnknownFormatConversionException", impact 4 System IDs, while

”java.lang.IllegalStateException” and ”java.lang.NegativeArraySizeException” affect 7 and 10 System IDs, respectively.

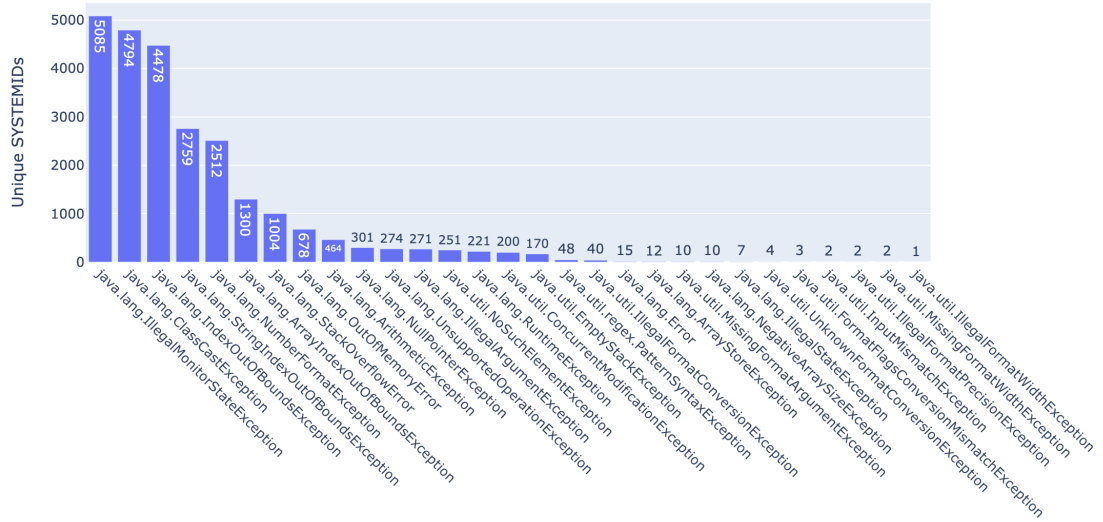
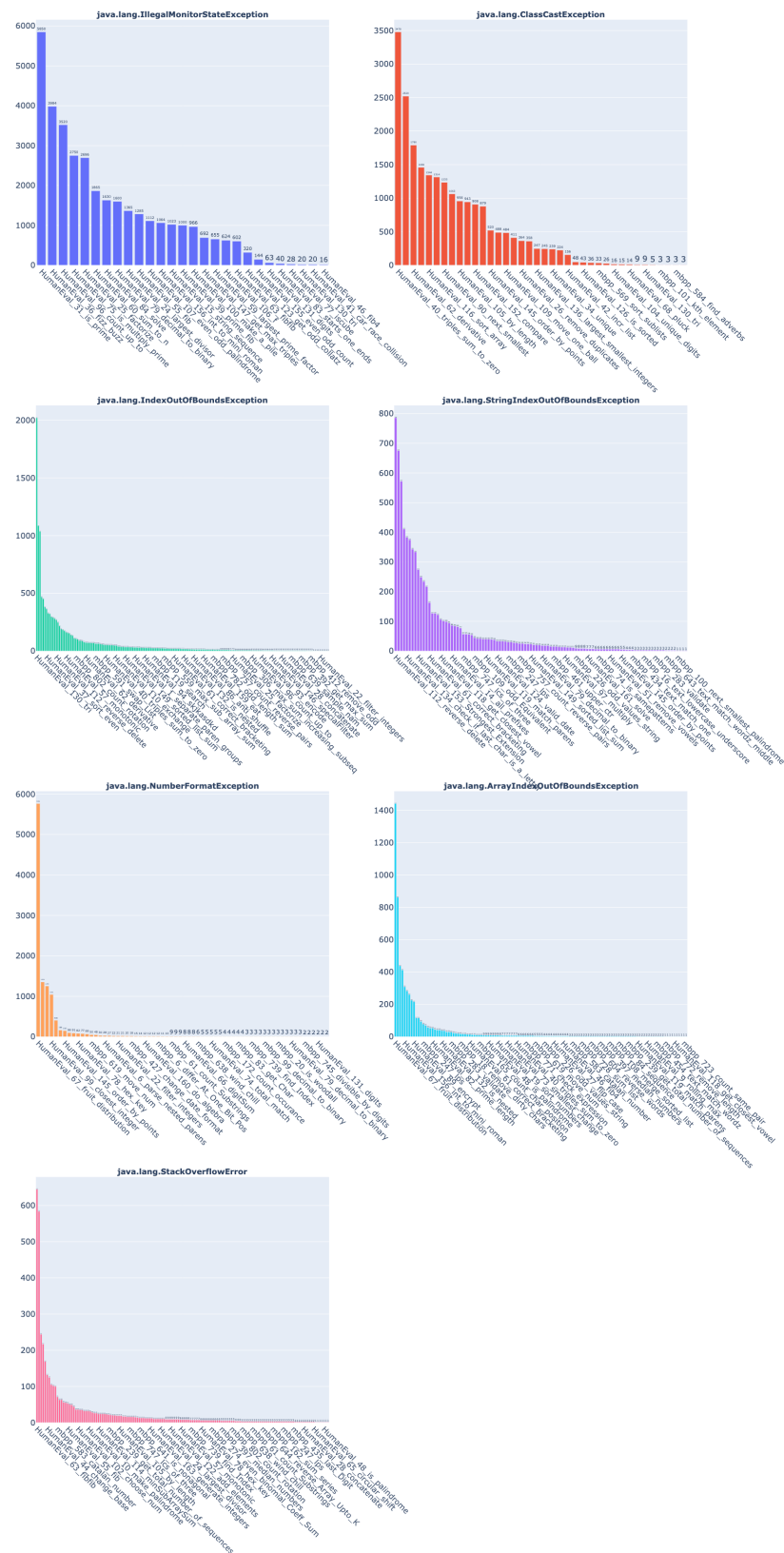


Figure 6.3.: Java exceptions across System IDs.

For further analysis, the top seven most frequently occurring errors, those appearing in more than 1000 System IDs, were selected and their distribution across functional abstractions are shown in Figure 6.4. For each of these java exceptions five System IDs were randomly selected to analyze the content of source code, even more, to understand the underlying causes. Therefore, inputs for each System ID were checked: whether they were valid or anomalies were present. Then their implementation code were checked to analyze how the source code handled these inputs, or if there were another issue with the implementation.



6.1.1. Analysis of `java.lang.IllegalMonitorStateException`

The most occurring Java exception across System IDs was "`java.lang.IllegalMonitorStateException`" (5085 System IDs). This exception is typically thrown when a thread tries to call a method like `wait()`, `notify()`, or `notifyAll()` on an object without holding the required monitor lock. This means that the thread is attempting to perform an operation that requires synchronization, but it has not acquired the necessary lock on the object. However, in the implementation of these selected System IDs none of the provided implementations explicitly use synchronized blocks or methods that require monitor locks. The exception is likely a result of deeper issues with how the testing framework processes inputs and executes the methods.

From the analysis of source codes of five selected System IDs where the "`java.lang.IllegalMonitorStateException`" occurred, a consistent pattern emerges: all functional abstractions tested under these System IDs had an extra input, represented as an empty dictionary `{}`, which deviated from their expected input signatures. Earlier in Section 4.3, it was observed that certain inputs had abnormal Value Types such as "`HumanEval_....Problem`", deviating from typical primitive types. This inconsistency aligns with the inclusion of empty dictionary inputs `{}`, suggesting a possible issue in the translation of Python benchmarks to Java code. These anomalies led to disruptions in function executions.

Although the specific functional abstractions and implementations vary, this additional input appears to be a systematic testing issue rather than an inherent flaw in the logic of the implementations.

For instance, in the case of "`HumanEval_63_fibfib`", which calculates the n -th element of a sequence similar to Fibonacci, the inputs included values such as 12, 1, 5, 2, 10, 8, and 14, with expected outputs like 274, 1, 4, 24, 80, and 927. Similarly, for "`HumanEval_31_is_prime`", which determines whether a number is prime, inputs such as 77, 17, 4, 61, and 255379 were provided. In the case of "`HumanEval_96_count_up_to`", which generates a list of prime numbers less than a given value, inputs included 101, 22, 0, 7, and 18.

In all these cases, the empty dictionary input led to type mismatches and disrupted execution, resulting in the observed exceptions. As for implementation, "`HumanEval_63_fibfib`" and "`HumanEval_31_is_prime`" would correctly return if

`{}` were not included. (see Listing 6.1 and 6.2). However, the implementation for "HumanEval_96_count_up_to" was flawed and would not have worked even if `{}` were removed. The function attempted to generate a list of integers up to the given input value but failed to include any logic for filtering prime numbers. Instead, it added all integers up to n to the result (see Listing 6.3).

```

1 public static int fibfib(int n) {
2     if (n <= 2) return 1;
3     return fibfib(n - 1) + fibfib(n - 2) + fibfib(n -
4         3);
5 }
```

Listing 6.1: Correct Implementation: "HumanEval_63_fibfib".

```

1 public static boolean isPrime(int n) {
2     if (n <= 1) return false;
3     if (n == 2) return true;
4     if (n % 2 == 0) return false;
5     for (int i = 3; i <= Math.sqrt(n); i += 2) {
6         if (n % i == 0) return false;
7     }
8     return true;
9 }
```

Listing 6.2: Correct Implementation: "HumanEval_31_is_prime".

```

1 public static ArrayList<Integer> countUpTo(int n) {
2     ArrayList<Integer> result = new ArrayList<>();
3     for (int i = 0; i < n; i++) {
4         result.add(i);
5     }
6     return result;
7 }
```

Listing 6.3: Incorrect Implementation: "HumanEval_96_count_up_to".

These examples highlight the importance of proper input validation and consistency in test case execution. The exception was triggered by unexpected "`{}`" inputs, leading to type mismatches and disruptions rather than direct misuse of synchronization methods. Ensuring correct input translation and preprocessing can help prevent such anomalies and improve reliability

6.1.2. Analysis of `java.lang.ClassCastException`

The next most occurring exception is "`java.lang.ClassCastException`" (4794). This exception is thrown when the Java runtime attempts to cast an object to a sub-

class or interface that it does not implement. For example, casting a `String` to an `Int` will result in this exception.

The analysis of source codes of five representative System IDs reveal that all provided inputs were lists of integers and appeared consistent with the expectations of the functional abstractions. No anomalies, such as mismatched data types or unexpected structures, were observed in the inputs. This suggests that the issue stems from the implementation logic rather than the provided data.

For example, in `HumanEval_40_triples_sum_to_zero`, the function attempted to determine whether three distinct elements in the input array summed to zero. Inputs such as `[2, 4, -5, 3, 9, 7]` and `[1, 3, 5, 0]` led to the exception. Although the function processed the array elements, its logic failed when fewer than three distinct elements were available, as the implementation assumed that the size of the set would always meet the requirement for comparison (see Listing 6.4).

```
1 public static boolean
   triplesSumToZero(ArrayList<Integer> l) {
2     Set<Integer> set = new HashSet<Integer>();
3     for(int i : l) {
4         set.add(Math.abs(i));
5     }
6     return set.size() == 3; // Fails if fewer than 3
   unique elements are present
7 }
```

Listing 6.4: Incorrect Implementation: "HumanEval_40_triples_sum_to_zero".

Another example is "HumanEval_37_sort_even", where the function was designed to sort elements at even indices while retaining the order of elements at odd indices. Inputs such as `[5, 3, -5, 2, -3, 3, 9, 0, 123, 1, -10]` caused the exception, because the logic for reconstructing the list after sorting failed when the sizes of the even and odd index lists did not match, leading to an out-of-bound error during iteration(see Listing 1 in appendix).

These examples highlight the importance of ensuring type safety when performing explicit casts. Issues arise when assumptions about data structures or element types do not hold, leading to casting failures. Proper type validation, use of generics, and structural checks can prevent such exceptions and improve reliability.

6.1.3. Analysis of java.lang.IndexOutOfBoundsException

The java.lang.IndexOutOfBoundsException appeared in 4478 System IDs. This exception is typically triggered when attempting to access an index outside the valid range of a collection or array. The analysis of source codes revealed recurring issues in implementation logic, particularly in edge case handling.

One notable example is "HumanEval_33_sort_third", a function designed to sort elements at indices divisible by three while retaining the rest unchanged. For inputs such as [1, 2, 4, 3, 5] and [4, 8, 5, 7, 3], the implementation assumed the presence of three consecutive indices in every iteration. When the array length was not a multiple of three, this assumption led to out-of-bounds access (see Listing 2 in appendix).

Another example is HumanEval_135_can_arrange, where the task was to find the largest index of an element not greater than or equal to its preceding element. For inputs like [4, 20, 8, 6, 9, 5, 3, 7], the implementation failed because it accessed arr.get(i - 1) without verifying if i > 0. This led to an attempt to access index -1, as shown in Listing 6.5.

```
1 public static long canArrange(ArrayList<Long> arr) {  
2     long largestIndex = -1;  
3     for (int i = 0; i < arr.size(); i++) {  
4         if (arr.get(i) > arr.get(i - 1)) { // Fails  
5             when i == 0  
6                 largestIndex = i;  
7         }  
8     }  
9     return largestIndex;  
}
```

Listing 6.5: Incorrect Implementation: "HumanEval_135_can_arrange".

The "java.lang.IndexOutOfBoundsException" observed in these examples highlights the need for robust input validation and careful handling of edge cases. Incorporating checks to ensure preconditions are met—such as verifying list sizes before iteration—would prevent such exceptions and enhance the reliability of these implementations

6.1.4. Analysis of "java.lang.StringIndexOutOfBoundsException"

The `java.lang.StringIndexOutOfBoundsException` was observed in 2579 System IDs, caused by the function attempting to access an index in a string that was out of its valid bounds. This exception occurs when an implementation assumes the presence of characters or indices in a string without proper validation.

One notable example is the "HumanEval_56_correct_bracketing" function, designed to determine whether every opening bracket `<` in a string has a corresponding closing bracket `>`. For inputs such as:

```
\u003c\u003c\u003e\u003c\u003e\u003e
```

the function failed to process the string correctly, resulting in the exception. The inputs were provided as Unicode-escaped representations of `<` and `>`, such as `"\u003c"` and `"\u003e"`, rather than raw characters `<` and `>`. This discrepancy in input format led to logical errors during iteration, as the function did not anticipate such encoded characters (see Listing 3 in appendix).

Another example is the `HumanEval.82_prime_length` function, which aimed to determine whether the length of a string is a prime number. For inputs like `"go"` and `"abcdcba"`, the function encountered the exception. The issue arose because the implementation attempted to access indices beyond the valid range of the string length, particularly when comparing adjacent characters or processing edge cases. This lack of proper boundary checks led to failures during runtime, as shown in Listing 6.6).

```
1 public static boolean primeLength(String string) {  
2     int length = string.length();  
3     for (int i = 1; i < length; i++) {  
4         if (string.charAt(i) == string.charAt(i + 1)) {  
5             continue;  
6         }  
7         if (string.charAt(i) < string.charAt(length))  
8         {  
9             return false;  
10        }  
11    }  
12    return true;  
}
```

Listing 6.6: Incorrect Implementation: "HumanEval.82_prime_length".

These examples highlight the need for thorough input validation and boundary checks when processing strings. Issues like Unicode-escaped characters and out-of-bound indices could be avoided by ensuring that inputs are correctly formatted and by incorporating safeguards to handle edge cases.

6.1.5. Analysis of "java.lang.NumberFormatException"

The "java.lang.NumberFormatException" occurred across 2512 System IDs. This exception is thrown when the Java runtime attempts to convert a string into a numeric type (e.g., `int`, `long`, or `double`) but encounters a string that does not conform to the expected format. Examples of such cases include strings with non-numeric characters or those representing numbers in unsupported formats (e.g., decimal values when parsing to `long`).

The abstraction "HumanEval_99_closest_integer" was designed to return the closest integer to a given string representation of a number. However, the implementation directly attempted to parse the input string using `Long.parseLong`, which only accepts whole numbers in a valid long format. Inputs such as "-15.5", "15.3", and "14.5" caused the exception because they contained decimal points. Since the function logic did not account for fractional values or invalid strings, these inputs resulted in runtime errors (see Listing 6.7).

The "HumanEval_67_fruit_distribution" function was tasked with calculating the number of mangoes in a basket, given a string representation of apples and oranges, and an integer for the total number of fruits. Inputs such as "1 apples and 100 oranges" failed because the implementation attempted to parse components of the string directly into numeric values using `Long.parseLong`. Since these string components included non-numeric characters like "apples" and "oranges", the parsing logic triggered this exception. A representative implementation is shown in Listing 6.8.

```
1 public static long closestInteger(String value) {  
2     long result = Long.parseLong(value); // Fails for  
3     strings with decimals  
4     return result;  
}
```

Listing 6.7: Incorrect Implementation: "HumanEval_99_closest_integer".

```
1 public static long fruitDistribution(String s, long n) {  
2     String[] arr = s.split(" ");  
3     long apples = Long.parseLong(arr[0]); // Fails if  
        arr[0] is not numeric  
4     long oranges = Long.parseLong(arr[2]); // Fails if  
        arr[2] is not numeric  
5     long mangoes = n - apples - oranges;  
6     return mangoes;  
7 }
```

Listing 6.8: Incorrect Implementation: "HumanEval_67_fruit_distribution".

These issues highlight the importance of implementing string validation and pre-processing to ensure input strings conform to the expected format, such as containing only numeric values. Additionally, the use of exception handling mechanisms, such as try-catch blocks, would allow the system to gracefully manage invalid inputs, providing meaningful feedback or fallback behavior.

6.1.6. Analysis of "java.lang.ArrayIndexOutOfBoundsException"

The `java.lang.ArrayIndexOutOfBoundsException` occurred in 1200 System IDs. This exception is thrown when a program attempts to access an array index that is either negative or exceeds the size of the array. Such errors are commonly caused by improper handling of edge cases or incorrect assumptions about input data size or structure.

One example is the implementation of "HumanEval_112_reverse_delete", which aims to remove characters from a strings based on characters specified in a second string and check if the resulting string was a palindrome. Inputs such as "abcd-edcba" and "v", and "dwik" and "w" caused the exception. The logic attempted to remove characters from the string but incorrectly calculated or assumed the index range during iteration, leading to out-of-bounds access (see Listing 4 in appendix).

Another example is the implementation of "HumanEval_116_sort_array", which aims to sort arrays based on the number of ones in their binary representation in ascending order. If two numbers have the same number of ones, they are sorted by their decimal value. However, issues arise due to incorrect logic in handling binary mappings and array indexing. Specifically, the count array is updated

with incorrect mappings, and iterations referencing out-of-bounds indices cause failures for certain inputs (see Listing 5 in appendix).

These examples underscore the importance of validating array indices and ensuring that operations involving array access are protected against invalid inputs. Proper bounds checking, comprehensive test cases, and clear handling of edge cases are critical for preventing such exceptions.

6.1.7. Analysis of "java.lang.StackOverflowError"

The "java.lang.StackOverflowError" occurred in 1004 System IDs. This error is typically triggered when a function's recursive calls exceed the program's stack limit, often due to improper base case handling or excessive recursion depth. Such errors highlight the importance of validating recursive logic to prevent infinite or excessive calls.

One example is the implementation of "HumanEval_83_starts_one_ends", which attempted to calculate the count of n-digit numbers starting or ending with the digit 1. Inputs such as 2 and 4 resulted in stack overflows due to a flawed recursive logic that failed to terminate properly when decreasing n (see Listing 6 in appendix).

Another notable case is "HumanEval_63_fibfib", which computed the n-th element of a sequence similar to the Fibonacci sequence. Inputs such as 10, 12, and 14 led to infinite recursion, as the implementation lacked a termination condition robust enough to handle larger inputs without exponential growth in function calls (see Listing 6.9).

A similar issue arose in "HumanEval_120_maximum", which aimed to compute the k maximum numbers in an array. Recursive calls to itself without effectively reducing the problem size for specific inputs, such as $[-3, 2, 1, 2, -1, -2, 1]$ with $k = 1$, resulted in stack overflows (see Listing 7 in appendix).

These examples emphasize the critical need for efficient recursive logic and proper base case handling to avoid excessive recursion. Iterative solutions, memoization, or alternative data structures should be considered to optimize such algorithms and prevent stack overflows.

```

1 public static long fibfib(long n) {
2     if (n == 0 || n == 1) {
3         return 0;
4     }
5     return fibfib(n-1) + fibfib(n-2) + fibfib(n-3);
6     // Recursion depth increases exponentially, causing
7     // a stack overflow
8 }

```

Listing 6.9: Incorrect Implementation: "HumanEval_63_fibfib".

6.2. Analysis of Randoop exceptions

The Randoop dataset initially contained 13605 rows, with 228 unique Abstraction IDs, 5771 unique Sheet IDs, and 1549 unique System IDs. Upon exploring the dataset, it was observed that there was only one type of randoop exception: "randoop.util.TimeoutExceededException". Figure 6.5 shows the relationship between the number of unique System IDs and number of unique inputs for each Abstraction ID.

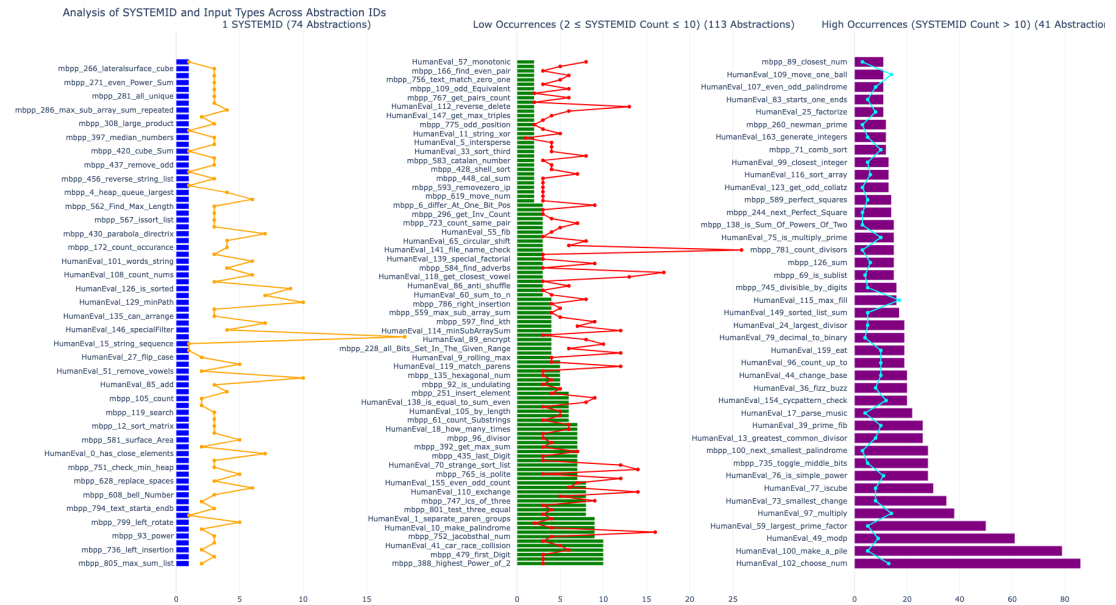


Figure 6.5.: Relationship between the number of unique SYSTEMIDs and number of unique inputs for each ABSTRACTIONID: Randoop exceptions.

Unlike Java exceptions, which had 29 different exceptions, there is only one type of randoop exception, therefore the abstraction "HumanEval_102_choose_num" was selected to be analyzed further, as it has the highest number of occurrences with 86 unique System IDs. This abstraction takes two positive integers, x and y and returns the largest even integer within the inclusive range $[x, y]$. If no such number exists, the function is expected to return -1. For example:

- `chooseNum(12, 15)` should return 14, the largest even number between 12 and 15.
- `chooseNum(13, 12)` should return -1 since no valid even number exists.
- `chooseNum(6, 29)` should return 28, the largest even number in that range.

There are thirteen unique inputs like '12354', '33', '12', '13', '6', '29', '546', '15', '5234', '5233', '7', '10', '27', which formed pairs, as the this abstraction takes input pairs. This means that there were not invalid inputs. In addition, there were five unique oracle outputs, including: -1, 28, 12354, 14, and 546.

Furthermore, five random System IDs were chosen for their corresponding source code analysis in order to investigate potential causes of the "Randoop.util.Timeout ExceededException". As a result several recurring issues were found that led to timeout exceptions:

- **Infinite Loops:** Most implementations use **while** loops to identify the largest even number. However, poorly defined or inconsistent termination conditions often result in infinite loops. For instance, when the variables used to narrow the range (even and odd) fail to converge or oscillate between values, the function cannot terminate within the allocated time, causing the exception (see Listing 8 in appendix).
- **Redundant Operations:** Many implementations perform unnecessary recalculations, such as repeatedly computing mid-points or making redundant adjustments to variables like even and odd. These operations add computational overhead without contributing to a faster or more accurate solution. In some cases, nested loops further exacerbate inefficiency, making the function slower and more prone to timeouts (see Listing 9 in appendix).
- **Edge Case Failures:** Handling of edge cases is inconsistent across implementations. For example, input ranges without even numbers or cases where x

$> y$ often cause the function to either loop indefinitely or produce incorrect results. These edge case failures highlight gaps in the robustness of the logic. (See Listing 10 in appendix).

- **Overly Complex Logic:** Some implementations introduce nested loops or overly complex logic to adjust the range. Instead of simplifying calculations, these structures delay convergence and increase the likelihood of timeout exceptions. For instance, repeatedly incrementing and decrementing variables within nested loops introduces unnecessary complexity. (See Listing 11 in appendix).
- **Inconsistent Mid-Point Logic:** Many implementations rely on mid-point calculations to narrow the range. However, incorrect updates to variables based on the mid-point often result in oscillating values that prevent the loop from terminating. This issue is especially prominent in scenarios with small ranges or when the range does not contain even numbers. (See Listing 12 in appendix).

6.3. Analysis of Null rows

In the null rows dataset, there were a total of 31256 rows across 131 unique Abstraction IDs, 31256 Sheet IDs and 4516 unique System Ids. Figure 6.6 shows the number of unique System Ids and unique inputs for each ABSTRACTIONID.

To investigate the cause of null rows, ten random System Ids were selected from three abstractions with the highest number of unique System Ids, including: "HumanEval_20_find_closest_elements", "HumanEval_155_even_odd_count", "HumanEval_111_histogram". For all of these abstractions, the inputs were valid and matched their expected format.

Upon examining the source code for these abstraction IDs, it was observed that all implementations were empty templates with no meaningful logic. Each implementation contained a placeholder comment such as `// TODO: Implement this method` and a return null statement within the method. A typical example of the generated code is as follows:

```

1 public static Pair<Float, Float>
2   findClosestElements(ArrayList<Float> numbers) {
3   // TODO: Implement this method.
4   return null;
5 }

```

Listing 6.10: Example of Empty Implementation in Null Rows Dataset.

This pattern was the same for all System IDs. LLMs might have struggled to generate proper code because the problem descriptions were too vague or the model didn't have enough examples in its training data. It might also be due to the model using a fallback approach when it is unsure what to do.

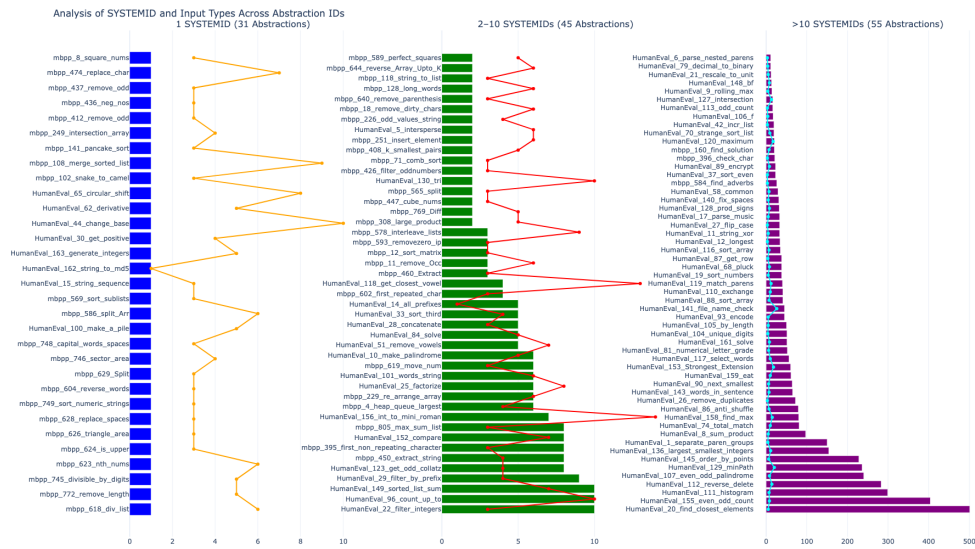


Figure 6.6.: Relationship between the number of unique SYSTEMIDs and number of unique inputs for each abstraction: Null rows.

6.4. Analysis of Nan rows

The Nan dataset contained 81 rows were identified, distributed across 7 unique Abstraction IDs, 36 Sheet IDs, and 33 System IDs. The original outputs were, specifically ”_NOT_EXECUTED_”, and their corresponding RAWVALUE entries were uniformly ”None”. The Figure 6.7 shows the relationship between the number of unique System IDs and number of unique inputs for each Abstraction ID.

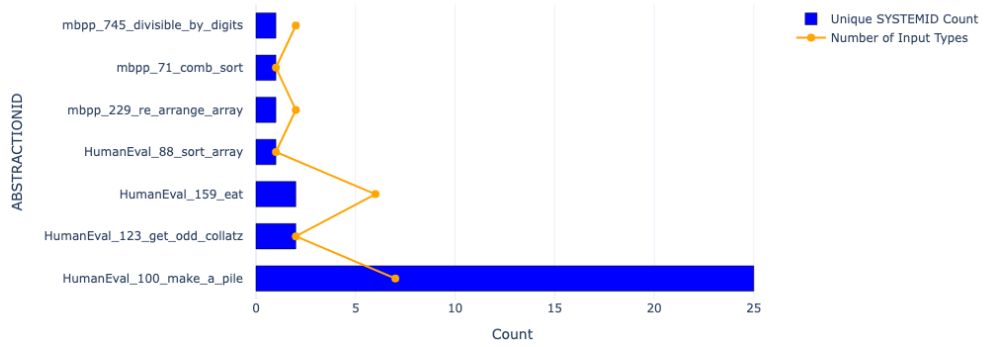


Figure 6.7.: Relationship between the number of unique System IDs and number of unique inputs for each Abstraction ID: Nan rows.

During the validation of inputs, it was observed that five rows contained `"_NOT_EXECUTED_"` as their input. Among these, four were linked to `"HumanEval_100_make_a_pile"` and one row was associated with `"HumanEval_123_get_odd_collatz"`. To investigate these cases, the oracle outputs of the affected SHEETIDs were compared, and expected inputs were deduced. It was determined that three cases should have had an input of `"5"` (two for `"HumanEval_100_make_a_pile"` and one for `"HumanEval_123_get_odd_collatz"`) and two cases for `"HumanEval_100_make_a_pile"` should have had inputs of `"6"` and `"8"`.

An additional anomaly was observed where an input included the value `"6"` along with an empty dictionary `"{}"`, despite the abstraction expecting only a single positive integer. This likely resulted from a translation error during data preparation, as the correct input should have been solely `"6"`.

Subsequently, five System IDs were selected randomly for implementation analysis, excluding those associated with `"_NOT_EXECUTED_"` inputs to focus on cases with valid inputs. These System IDs included three linked to `"HumanEval_100_make_a_pile"` and two associated with `"HumanEval_159_eat"`. As a result of analysis several potential causes were found:

- **Infinite Loops:** In `"HumanEval_100_make_a_pile"`, the loop termination condition was either incorrectly defined or overly broad. This resulted in excessive iterations, leading to the function exceeding computational limits (See Listing 13 in appendix).
- **Unvalidated Inputs:** While the inputs analyzed were valid, the code for

"HumanEval_159_eat" lacked robust handling for edge cases, particularly when inputs approached boundary conditions such as zero or exceeding stock limits. This lack of validation could result in unexpected behavior or execution failure (See Listing 14 in appendix).

- **Inefficient Logic:** Both abstractions included redundant or overly complex operations. For example, in "HumanEval_159_eat", unnecessary recalculations in nested loops increased computational overhead, while "HumanEval_100_make_a_pile" performed unnecessary iterations, delaying termination (See Listing 15 in appendix).
- **Resource Constraints:** The computational cost of the loops, particularly in cases involving large ranges or repeated operations, likely caused execution to be prematurely halted due to resource constraints or timeouts (See Listing 16 and 17 in appendix).
- **Translation Errors:** The presence of invalid inputs, such as the combination of an integer and an empty dictionary in earlier observations, highlights potential translation issues during data preparation. Although not present in these selected SYSTEMIDs, such errors underscore challenges in maintaining data integrity (See Listing 18 in appendix).

6.5. Valid Non-matching Dataset

The valid non-matching dataset contained 667613 rows, accounting for approximately 82% of the complete non-matching data set. This dataset includes 438 unique Abstraction IDs, 667613 unique Sheet IDs, and 177391 unique System IDs. Due to the large scale of the data, analyzing each functional abstraction individually is not feasible. Therefore, for further analysis, only some abstractions were prioritized based on the diversity of their distinct outputs.

As a first step, the number of distinct outputs for each abstraction was counted, revealing significant variation in output diversity, as shown in Figure 6.8. Some abstractions produced hundreds of distinct outputs, while others had only one. Since the number of distinct outputs per functional abstraction can reach up to 899, priority was given to functional abstractions with minima variability, as they might provide clear examples of deterministic behavior across test cases. In

total, seven functional abstractions that had only 1 distinct output was selected: "HumanEval_87_get_row", "mbpp_787_text_match_three", "mbpp_737_check_str", "mbpp_91_find_substring", "mbpp_605_prime_num", "mbpp_798_sum", and "mbpp_127_multiply_int".

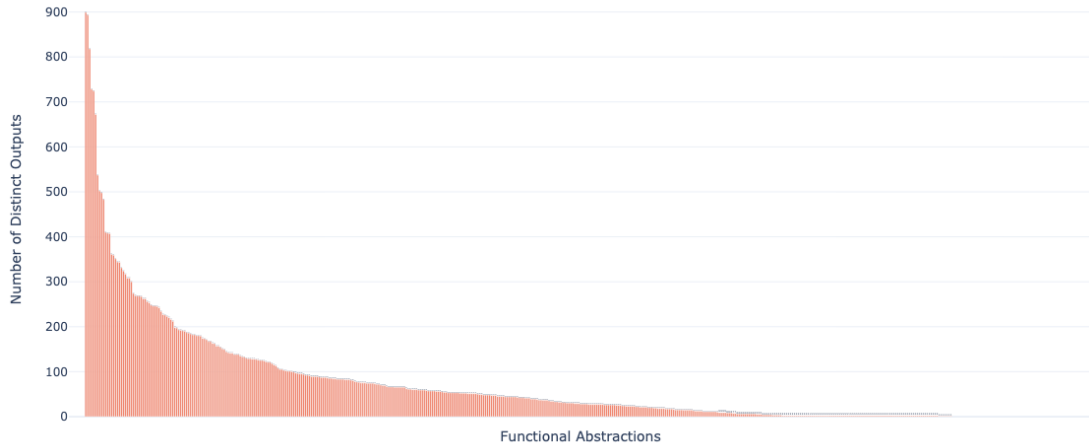


Figure 6.8.: Distinct output counts per functional abstraction.

Furthermore, as part of the analysis of abstractions with minimal output diversity, a systematic approach was followed to investigate the reasons behind their consistent mismatches. The analysis was divided into two phases.

The first phase includes examining all the following aspects for each abstraction:

- Input Validation: Ensuring that the inputs provided were valid and matched the abstraction's intended functionality.
- Types of Inputs: Identifying the variety and patterns of inputs used in the test cases.
- Number of unique System IDs: Determining the number of unique System IDs contributing to the mismatch to understand whether the issue was isolated or widespread.
- Oracle vs Original Outputs: Comparing the oracle outputs with the original outputs to pinpoint the nature of the mismatch.

The second phase delved into source code analysis. The source code of each System ID was retrieved using SORL_data to inspect the associated source code.

The input-output pairs were verified across three sources: the original benchmark, which defines the problem and its constraints; the translated benchmark, ensuring that the test cases were correctly adapted during the translation process; and the retrieved source code, to check for consistency with the benchmarks and expected behavior. Following this, the implementation in the retrieved source code was examined for logical errors, issues in edge case handling, and deviations from benchmark specifications. For some functional abstractions that had too many System IDs, only five of them were selected randomly for further analysis.

6.5.1. Analysis of HumanEval_87_get_row

This functional abstraction involves processing a two-dimensional dataset represented as a nested list. Unlike conventional matrices, the rows in this dataset may contain a different number of columns. Given a nested list, `lst`, and an integer, `x`, the goal is to locate all occurrences of `x` within the dataset and return a list of pairs representing their coordinates, (row, column). The row and column indices start from 0. The output list of coordinates should be sorted first by row indices in ascending order. For rows with multiple occurrences of `x`, the coordinates within the row should be sorted by column indices in descending order.

The expected output for inputs "[[1]]" and 2" is an empty list [], as the target value does not exist in the input. However there was one System ID with output: [{"val0":0,"val1":1,"valueArray":[0,1],"valueList":[0,1]}]. The original Python prompt includes a test case that explicitly defines this input-output pair: `assert candidate([[1]], 2) == []`. Similarly, the Java translation of the benchmark includes a corresponding assertion, as shown in Listing 6.11. The source code retrieved via System ID also includes this test case in the main method.

This ensures that the behavior for the input "[[1]]" and 2 is explicitly tested in the provided Java implementation. However, despite the presence of these assertions, the actual execution of the method with the same input produced an incorrect result.

Furthermore, the source code analysis indicates that this observed behavior may have been caused by logical flaws in the implementation of the `getRow` method. As shown in Listing 6.12, the implementation included an unnecessary condition

and a premature return that led to the incorrect output. This condition checks whether the first row of the input contains exactly one element. For the input `[[1]]`, this condition evaluates to true. As a result, a pair (0, 1) is added to the result list `ret`, and the method exits prematurely without further processing the input. This behavior directly contradicts the expected functionality of the method, which should iterate through the entire input and only add coordinates when the target value is found.

```

1 assert(getRow(
2     new ArrayList<ArrayList<Long>>(Arrays.asList(
3         (ArrayList<Long>) new
4             ArrayList<Long>(Arrays.asList((long)11))
5     )),
6     (21)
7 ).equals(new ArrayList<Pair<Long,
8     Long>>(Arrays.asList())));

```

Listing 6.11: Java implementation of " `[[1]], 2) == []`".

```

1 if (lst.get(0).size() == 1) {
2     ret.add(new Pair<>(0L, lst.get(0).get(0)));
3     return ret;
4 }

```

Listing 6.12: Problematic Condition in `getRow`.

Another contributing factor to the incorrect behavior is the serialization of the `Pair` object. Instead of returning an empty list, the method returns a serialized object containing the pair (0, 1) along with additional fields generated by the serialization library. The serialized output appears as:

```
[{"val0":0,"val1":1,"valueArray":[0,1],"valueList":[0,1]}]
```

Here, "val0" represents the row index, "val1" represents the column value, and fields such as `valueArray` and `valueList` are artifacts of the serialization process.

These issues highlight the discrepancy between the intended functionality of the `getRow` method and its actual implementation. The unnecessary condition introduced flawed logic, leading to premature termination of the method, while the serialization process compounded the error by producing unexpected fields in the output. This behavior underscores the need for rigorous testing and verification to ensure consistency between specifications, assertions, and implementations.

6.5.2. Analysis of mbpp_787_text_match_three

This functional abstraction "mbpp_787_text_match_three" creates a function that matches a string containing the letter 'a' followed by exactly three 'b'. The task requires validating whether the input string adheres to this specific pattern.

Unique inputs for non-matched dataset were: "abbbba" and "caacabbbba". The expected output was "true", however, the original output was "false", failing to identify matches. This issue involved 327 unique System IDs, which indicates a widespread problem affecting multiple source codes.

The Java version of the benchmark has all test cases from the original benchmark and introduces an additional test case: `assert(textMatchThree("caacabbbba") == true)`, which check whether the function can identify the specified pattern as a substring within a larger string.

As a result of analyzing the implementation of source codes, it was found that all source codes were consistent in test cases, however there were significant discrepancies were observed in the regex implementation. The observed patterns revealed varying degrees of deviation from the intended logic. These included:

- **Overly Permissive Patterns:** Some implementations allowed invalid matches, such as strings that lack the required 'a' or contain non-contiguous occurrences of 'b' (see Listing 6.13).
- **Overly Restrictive Patterns:** Other implementations restricted valid matches by requiring specific quantities or sequences of 'a' and 'b', excluding cases like "abbb" (see Listing 6.14).
- **Misinterpretation of Problem Requirements:** In certain source codes, the regex reflected a misunderstanding of the task, treating 'b' as repeatable groups or introducing unnecessary flexibility in matching patterns (see Listing 6.15).


```
1 public static boolean textMatchThree(String text) {  
2     return text.matches("a*b{3}");  
3 }
```

Listing 6.13: Overly Permissive Regex: "mbpp_787_text_match_three".

```
1 public static boolean textMatchThree(String text) {  
2     return text.matches("a{3}b");  
3 }
```

Listing 6.14: Overly Restrictive Regex: "mbpp_787_text_match_three".

```
1 public static boolean textMatchThree(String text) {  
2     return text.matches("a+b+b+b+");  
3 }
```

Listing 6.15: Misinterpreted Regex: "mbpp_787_text_match_three".

6.5.3. Analysis of mbpp_737_check_str

The functional abstraction `mbpp_737_check_str` is designed to verify whether a given string starts with a vowel. The functionality relies on a regular expression (regex) pattern to check the first character of the string against a predefined set of vowels. If the string starts with a vowel, the function returns a positive result ("Valid" in Python, "true" in Java); otherwise, it returns a negative result ("Invalid" in Python, "false" in Java).

For the non-matching dataset, 129 unique System IDs were identified, each representing a distinct implementation of the functional abstraction. Among these System IDs, two unique inputs were observed: "annie" and "Else". Their expected output was "true", however all these 129 implementations generated "false". Also, the Java benchmark has the same functional intent as the original benchmark but alters the outputs to true/false and includes the same test cases.

The incorrect behavior stems from flaws in the regex patterns used in the implementations. For example, some implementations only check if the string consists of a single vowel, rather than verifying if it starts with one (see Listing 6.16). Other implementations enforce that the entire string must be composed of vowels, ignoring any multi-character strings that do not match this overly restrictive condition (see Listing 6.17). These logical errors result in mismatched outputs for valid inputs.

```
1 public static boolean checkStr(String string) {  
2     return string.matches("[aeiouAEIOU]");  
3 }
```

Listing 6.16: Flawed Regex: "mbpp_737_check_str".

```
1 public static boolean checkStr(String string) {  
2     return string.matches("^[aeiou]+$");  
3 }
```

Listing 6.17: Overly Restrictive Regex: "mbpp_737_check_str".

6.5.4. Analysis of mbpp_91_find_substring

The functional abstraction "mbpp_91_find_substring" is designed to determine whether a specified substring exists within any of the strings in a given list. The function takes two inputs: a list of strings and a substring to search for. If the substring is found in at least one string in the list, the function is expected to return "true"; otherwise, it returns "false".

The Python benchmark uses a generator expression combined with the "any" function to efficiently evaluate the presence of the substring. The translated Java benchmark maintains this functional intent and includes identical test cases to verify correctness. However, one implementation in the non-matching dataset failed to detect substrings correctly. It consistently returned "false" for cases where the expected output was "true." For example, the functional abstraction incorrectly returned "false" when given the input list ["red", "black", "white", "green", "orange"] and the substring "ack."

The root cause of this issue lies in the flawed logic of the implementation. Instead of checking whether the substring exists within any string in the list, the function incorrectly verifies if the substring itself is a direct element of the list, as shown in Listing 6.18. This logic uses the "contains" method, which only checks for exact matches, failing to search for substrings within each string. This fundamental misinterpretation of the task requirements highlights the critical importance of aligning implementation logic with problem requirements, particularly when adapting benchmarks between programming languages.

```
1 public static boolean findSubstring(ArrayList<String>  
   str1, String sub_str) {  
2     if (str1.contains(sub_str)) {  
3         return true;  
4     }  
5     return false; }
```

Listing 6.18: Flawed Logic: "mbpp_91_find_substring".

6.5.5. Analysis of mbpp_605_prime_num

The functional abstraction "mbpp_605_prime_num" is designed to determine whether a given integer is a prime number. The function is expected to return "true" for prime numbers and "false" otherwise. Notably, negative numbers cannot be prime, and the function should correctly handle such inputs by returning "false". There were in total 12 System IDs exhibited incorrect behavior when tested with the input "-1010". The expected output for this input was "false", as negative numbers are not considered prime. However, these System IDs consistently returned "true", indicating a failure to account for negative numbers.

The original Python benchmark implements the functionality by first checking whether the number is greater than or equal to 1. If this condition is satisfied, the function iterates through potential divisors to determine primality. The corresponding Java benchmark maintains functional alignment and includes the same test cases. Both benchmarks specify that the input -1010 should return "false". The incorrect behavior observed in some implementations can be attributed to a lack of input validation. Instead of verifying whether the input is positive before performing primality checks, these implementations directly proceed with their logic. As shown in Listing 6.19, this approach incorrectly classifies negative numbers as prime by default, as the loop for checking divisors is not entered when the input is negative.

This example highlights the importance of including explicit checks for edge cases, such as negative numbers, to ensure the function adheres to its intended requirements. Without such validation, the logic assumes all inputs are valid candidates for primality checks, leading to incorrect outputs. By contrast, the benchmark implementation properly handles these edge cases, ensuring negative numbers are excluded before further processing.

```
1 public static boolean primeNum(long num) {  
2     boolean isPrime = true;  
3     for (int i = 2; i <= num / 2; i++) {  
4         if (num % i == 0) {  
5             isPrime = false;  
6             break;  
7         }  
8     } return isPrime;  
9 }
```

Listing 6.19: Flawed Logic: "mbpp_605_prime_num".

6.5.6. Analysis of mbpp_798_sum

The functional abstraction "mbpp_798_sum" is designed to compute the sum of all elements in a given array or list of numbers. It is expected to return the total sum of the elements as a single numeric value. For example, the input lists [0,1,2], [1,2,3] and [15,12,13,10] should return the outputs 3, 6, and 50 respectively.

The analysis of the non-matching dataset revealed a single SYSTEMID that failed to compute the correct sums for any of the test inputs. Instead of returning the expected sum, the implementation consistently returned 0 regardless of the input.

The original Python benchmark correctly implements the sum functionality by iterating through the elements of the input list and adding them to a cumulative total. This implementation ensures that all elements in the list are processed, and it properly handles a variety of inputs, including those containing zeros. The Java translation of the benchmark maintains functional equivalence and includes the same set of test cases.

Despite valid test cases and a correct Java benchmark specification, this SYSTEMID failed to produce the correct outputs, highlighting a fundamental issue with the implementation logic. The implementation explicitly returns "0" without performing any calculations, completely bypassing the logic required to compute the sum. Consequently, regardless of the input, the output remains "0" (see Listing 6.20).

As a result, this example highlights the challenges of ensuring that all System IDs adhere to benchmarks and specifications. Even with valid test cases and a clear problem description, errors in implementation, can lead to consistent failures.

```
1 public static long Sum(ArrayList<Long> arr) {  
2     // Write your code here  
3     return 0;  
4 }
```

Listing 6.20: Incorrect Implementation: "mbpp_798_sum".

6.5.7. Analysis of mbpp_127_multiply_int

Similar to "mbpp_798_sum", the functional abstraction "mbpp_127_multiply_int" also consistently returned 0 for all the five selected System IDs initially analyzed. The functional abstraction is designed to compute the product of two integers without using the multiplication operator (*), instead relying on iterative or recursive addition. For example, the input pair (10, 20) is expected to return 200, (5, 10) should yield 50, and (4, 8) should produce 32.

Given the consistent failure across the five sampled System IDs, a deeper analysis of all 22 System IDs associated with this functional abstraction was conducted. This analysis revealed that all implementations returned 0 for every test input, regardless of the values, indicating a systemic issue in the dataset.

The original Python benchmark implements the functionality correctly by recursively or iteratively adding the first number to itself based on the value of the second number. This ensures accurate results across various scenarios, including negative multipliers. The Java benchmark maintains functional equivalence and includes the same test cases to validate the correctness of the implementations. However, all faulty System IDs failed to perform any multiplication logic and simply returned 0 without processing the inputs, as shown in Listing 6.21.

```
1 public static long multiplyInt(long x, long y) {  
2     // TODO: Implement this function  
3     return 0;  
4 }
```

Listing 6.21: Incorrect Implementation: "mbpp_127_multiply_int".

7. Final Results

This chapter presents the final results of the study. First, Section 7.1 provides a comprehensive overview of the developed taxonomy. Then, Section 7.2 focuses on the analysis of matched data, where clusters of functionally similar abstractions are examined, highlighting different types of behavioral and contextual relationships. Section 7.3 presents the results for non-matching data, analyzing cases where outputs deviated from expected oracle results. This section is further divided into two parts: Section 7.3.1 investigates errors and exceptions, such as Java runtime errors and execution failures. Section 7.3.2 focuses on valid but non-matching outputs.

7.1. Complete Taxonomy Overview

The goal of this research was to develop a taxonomy for software responses to evaluate functional equivalence, identify behavioral relationships, and address challenges in testing LLM generated code. The taxonomy categorizes functional abstractions into matched and non-matching clusters, each further classified into subcategories that help distinguish different types of software responses.

The final taxonomy is shown in Figure 7.1. At the top level (red), the two main branches: Matching Data and Non-matching Data are defined. Beneath them, the second level (blue) represents specific response categories within each branch, such as different types of equivalence, contextual relationships, and error classifications. Finally, the third level (yellow) differentiates responses based on their connectivity, indicating varying degrees of relationship strength between responses.

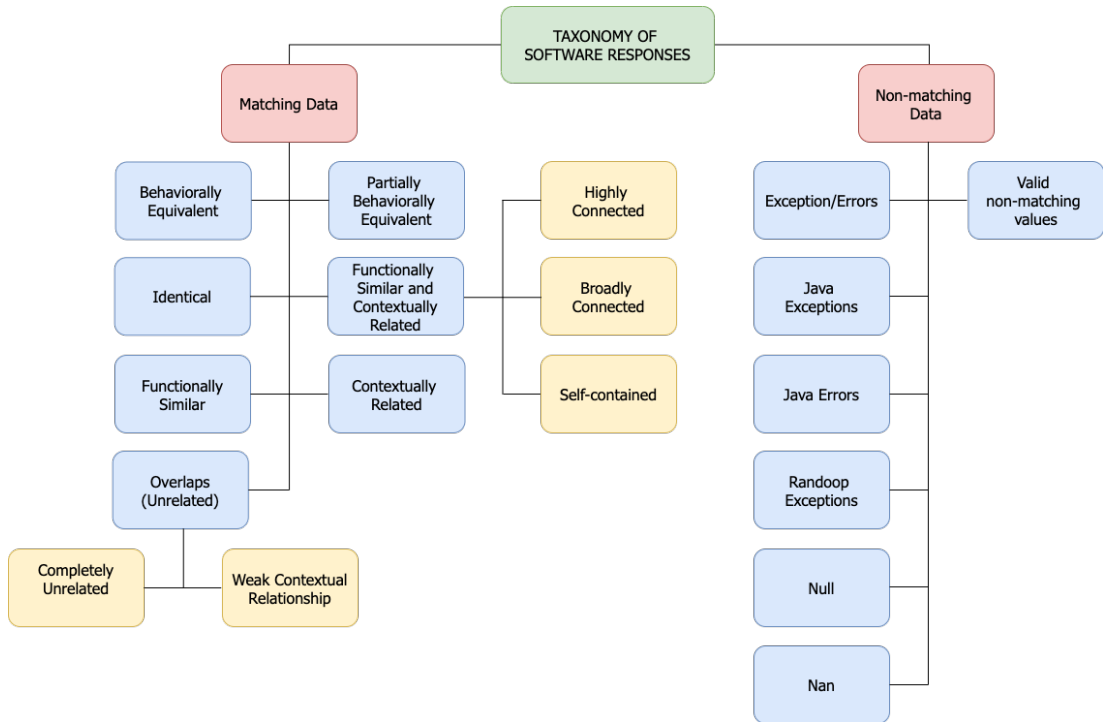


Figure 7.1.: Taxonomy of software responses.

7.2. Results for Matched Data Analysis

The matched data analysis revealed various relationships between functional abstractions, highlighting their behavioral equivalence, functional similarities, and contextual connections. These relationships were further grouped into distinct categories, as summarized in Table 7.1. Each category reflects a different aspect of the matched responses, ranging from complete behavioral equivalence to contextual relationships without functional similarity. The complete categorization of abstraction clusters by behavioral relationships is shown in A1 in appendix.

Another notable finding was the identification of errors in oracle outputs within certain clusters. Specifically, in the cluster containing `mbpp_790_even_position` and `mbpp_775_odd_position`, an inconsistency was observed where the oracle incorrectly labeled the output for the input `[2,3]` as false, despite the input satisfying the expected parity constraints. Additionally, no explicit assertion for this case was present in both the original and translated benchmarks, indicating a potential misclassification. Such errors in oracle outputs highlight the challenges in assessing behavioral equivalence and emphasize the need for careful validation of

benchmark datasets.

Relationship	Features	Common Observations	Representative Clusters
Behaviorally Equivalent	Identical input-output behavior for specific test cases but differing implementations.	Behaviorally equivalent outputs are observed in overlapping test cases, but implementations differ significantly.	mbpp_458_rectangle_area, mbpp_127_multiply_int
Partially Behaviorally Equivalent	Similar logic with variations under specific input conditions; partial output equivalence.	Behavioral differences occur due to edge cases or oracle errors; domain-specific focus leads to variations.	mbpp_790_even_position, mbpp_775_odd_position; mbpp_736_left_insertion, mbpp_786_right_insertion
Functionally Similar and Contextually Related	Clusters addressing related tasks with overlapping goals but distinct implementations.	Contextual connections across abstractions, focusing on tasks like sequence analysis, sorting, or pattern matching. Includes <i>Highly Connected Clusters</i> , <i>Broadly Connected Clusters</i> , and <i>Self-Contained Clusters</i> .	Their representatives are shown below.
<i>Highly Connected Clusters</i>	A small number of tightly connected clusters where all functional abstractions are exclusive to these clusters.	Functional abstractions appear only in these clusters and not in others, indicating tightly-knit relationships.	mbpp_787_text_match_three, mbpp_756_text_match_zero_one, mbpp_285_text_match_two_three, mbpp_434_text_match_one
<i>Broadly Connected Clusters</i>	A larger number of interconnected clusters with some shared functional abstractions across clusters.	Overlapping functional abstractions connect these clusters, creating a broader network of relationships.	HumanEval_126_is_sorted, mbpp_751_check_min_heap, mbpp_472_check_Consecutive, mbpp_281_all_unique; mbpp_760_unique_Element, mbpp_19_test_duplicate
<i>Self-Contained Clusters</i>	Isolated clusters with functional abstractions that do not appear in other clusters.	Functional abstractions are exclusive to each self-contained cluster, with no connections to other clusters.	HumanEval_63_fibfib, HumanEval_46_fib4; mbpp_454_text_match_wordz, mbpp_643_text_match_wordz_middle
Functionally Similar Clusters	Clusters exhibiting similar computational techniques with differing objectives.	Functional similarity arises from shared computational operations, but clusters lack strong contextual ties.	mbpp_797_sum_in_range, HumanEval_53_add; mbpp_296_get_Inv_Count, HumanEval_73_smallest_change
Identical Abstractions	Clusters formed by identical functional abstractions with the same outputs and logic.	Identical abstractions appear across data splits, suggesting intentional benchmarking overlaps.	mbpp_591_swap_List, mbpp_625_swap_List
Unrelated Clusters	Clusters with weak or no functional and contextual overlap, often coincidental matches.	Includes <i>Completely Unrelated Clusters</i> and <i>Unrelated Clusters with Weak Contextual Relationship</i> .	Their representatives are shown below.
<i>Completely Unrelated Clusters</i>	No overlap in logic, purpose, or behavior.	Shared input-output pairs are purely coincidental; tasks are fundamentally distinct.	HumanEval_162_string_to_md5, HumanEval_111_histogram; mbpp_762_check_monthnumber_number, mbpp_406_find_Parity
<i>Unrelated Clusters with Weak Contextual Relationship</i>	Weak connections through shared data types or superficial overlaps.	Weak contextual overlap from shared input types, but no meaningful functional, behavioral, or contextual equivalence.	mbpp_404_minimum, HumanEval_97_multiply; mbpp_741_all_Characters_Same, mbpp_113_check_integer

Table 7.1.: Cluster Analysis Results for Matched Data.

7.3. Results for Non-matching Data Analysis

7.3.1. Exception and Errors

This subsection analyzes errors and exceptions in the non-matching dataset, focusing on java exceptions, randoop exceptions, null rows, and nan rows. The Table 7.2 below highlights frequency of exceptions across unique System IDs, summarizes the key characteristics and insights/observations identified in the non-matching dataset,

Category	Number of SYS-TEMIDs	Key Characteristics	Insights and Observations
Java Exceptions (Total)	24405	Includes 29 distinct Java exception types.	A subset of seven frequently occurring exceptions (each observed in more than 1,000 SYSTEMIDs) was analyzed. The primary failure causes included unvalidated inputs, poor edge case handling, translation inconsistencies, and flawed implementations. These exceptions often indicated structural weaknesses rather than specific functional abstraction failures.
IllegalMonitorStateException	5085	Caused by unexpected empty dictionary {} inputs, leading to execution failures.	The exception arose not due to improper synchronization but rather because malformed or unrecognized inputs disrupted execution. Proper input translation and preprocessing are necessary to mitigate such anomalies.
ClassCastException	4794	Occurred due to incorrect type assumptions in casting operations.	Observed primarily in cases where structural checks were missing, leading to failures when dynamically typed elements were cast incorrectly. Introducing generics and enforcing stricter type validation would reduce such issues.
IndexOutOfBoundsException	4478	Triggered by out-of-range access to lists and arrays.	Occurred when input lists were smaller than expected, often due to missing validation steps. Implementing boundary checks before iteration would prevent these failures.
StringIndexOutOfBoundsException	2579	Caused by string operations that exceeded valid index ranges.	Found in abstractions processing substrings or Unicode characters, highlighting the need for robust validation before slicing or indexing operations.
NumberFormatException	2512	Caused by invalid numeric parsing from improperly formatted strings.	Errors occurred when functions attempted to convert strings to numbers without first ensuring their validity. Preprocessing steps should be introduced to validate input format before conversion.
ArrayIndexOutOfBoundsException	1200	Resulted from attempting to access array indices outside valid boundaries.	This issue was prominent in cases where dynamically sized lists were handled improperly. Proper bounds checking and defensive programming techniques should be implemented.
StackOverflowError	1004	Caused by unbounded recursion without proper termination conditions.	Observed in functions with deep recursive calls. Refactoring these implementations using iterative approaches or memoization could prevent stack overflows.
Randoop Exceptions	1549	One type of exception: <code>TimeoutExceededException</code> .	Randoop failures stemmed from infinite loops, redundant operations, and inefficient logic. Errors were primarily found in "HumanEval_102.choose_num," affecting 86 SYSTEMIDs. Poor termination conditions caused functions to oscillate indefinitely, leading to computational exhaustion.
Null Rows	4516	Placeholder implementations with <code>// TODO</code> comments and null outputs.	The analyzed SYSTEMIDs belonged to tasks such as "HumanEval_20.find_closest_elements," where no functional logic was present. These implementations returned null as they lacked any meaningful computation.
NaN Rows	33	Outputs were consistently <code>"_NOT_EXECUTED_"</code> , with <code>RAWVALUE = None</code> .	The NaN dataset contained only 81 rows, distributed across seven abstractions. Anomalies were observed, including translation errors, invalid inputs (e.g., an empty dictionary combined with an integer), and inefficiencies in loop termination and edge case handling.

Table 7.2.: Summary of Errors and Exceptions in the Non-Matching Dataset.

7.3.2. Valid Non-matching Outputs

This subsection focuses on the analysis of valid but diverging outputs in the non-matching dataset. These mismatches occur when the provided inputs are valid and adhere to the expected format, yet the outputs differ between the oracle and the system implementations. Unlike errors and exceptions, these cases reveal deeper issues related to implementation logic, algorithmic choices, and edge-case handling.

The analysis of non-matching outputs confirms that the inputs for all abstractions were valid, ruling out test case errors as the source of mismatches. Boolean mismatches, such as "true/false", frequently occurred in abstractions like "mbpp_787_text_match_three" and "mbpp_737_check_str." While some mismatches, such as those in "HumanEval_87_get_row" and "mbpp_91_find_substring", were isolated to a single System ID, others, such as "mbpp_127_multiply_int" (22 System IDs), "mbpp_787_text_match_three" (327 System IDs), and "mbpp_737_check_str" (129 System IDs), spanned multiple System IDs. These mismatches underscore implementation issues related to string matching, calculations, handling "true/false" values, and edge cases like negative numbers.

So far, no instances of translation loss have been found as the root cause of mismatches. The analysis shows that benchmark translations from Python to Java accurately preserved the functional intent and test cases.

The observed mismatches resulted solely from flawed implementations across various System IDs. These implementation issues included logical oversights and unimplemented placeholders:

- In "HumanEval_87_get_row", an unnecessary condition led to premature termination of logic, causing incorrect outputs.
- In "mbpp_787_text_match_three", incorrect regex patterns or overly flexible implementations caused mismatches in string detection logic.
- In "mbpp_737_check_str", overly restrictive or misinterpreted regular expressions led to mismatches for valid test cases.
- In "mbpp_91_find_substring", the use of incorrect methods like `contains` resulted in false negatives for valid substring checks.

-
- In "mbpp_605_prime_num", some implementations failed to validate negative inputs, incorrectly classifying them as prime.
 - In "mbpp_127_multiply_int" and "mbpp_798_sum", implementations consistently returned 0 for all test cases, indicating a complete absence of functional logic.

8. Discussion

This chapter reflects on the findings of the thesis, examining its limitations and outlining directions for future work. While the study successfully developed a preliminary taxonomy of response types for analyzing LLM-generated code, the scope of the analysis was limited by the scale of the dataset, time constraints, and the exploratory nature of the research. The limitations highlight areas where the methodology and results could be refined or expanded. Building on these insights, the future work section identifies key steps to advance this research, including extending the taxonomy, validating its applicability, and addressing challenges such as edge case handling, translation fidelity, and system-level analysis.

8.1. Limitations

The taxonomy developed in this thesis is preliminary and limited in scope. Only a subset of functional abstractions, test cases, and Java exceptions was analyzed to categorize response types. Specific subsets, such as seven Java exceptions, one Randoop exception, and select non-matching outputs, were chosen based on their frequency or variability in outputs. As a result, the taxonomy does not fully encompass the diversity of response types present in the dataset, limiting its completeness.

The scale of the dataset and the scope of the benchmarks also posed significant challenges. This study focused on the HumanEval and MBPP benchmarks, particularly their Java translations. While these benchmarks provide a solid foundation for analysis, they represent only a subset of programming challenges and may not generalize to other benchmarks or real-world development scenarios. Furthermore, the dataset’s large scale—comprising over 300,000 unique system outputs—made it impractical to comprehensively analyze all source codes, necessitating reliance on random sampling.

Translation-related biases and data integrity issues were another limitation. Observed anomalies, such as additional empty dictionaries in Java exception inputs, indicate potential translation losses during the Python-to-Java conversion by MultiPL-E or errors during data import into the LASSO platform. Such inconsistencies may introduce subtle biases or inaccuracies, which could impact the findings and conclusions drawn from the dataset.

The study’s analysis of implementation errors and logical flaws did not exhaustively cover edge cases for functional abstractions. Time and resource constraints restricted the ability to investigate nuanced failure modes fully, limiting insights into how implementations handle rare or complex input conditions. Additionally, the focus remained on input-output relationships, leaving the intermediate computational processes largely unexplored. A deeper analysis of these processes could have revealed additional patterns or challenges in LLM-generated code.

The development of the taxonomy relied heavily on manual clustering and relationship evaluations among functional abstractions. While this approach allowed for detailed and nuanced observations, it is inherently time-intensive and susceptible to human error and subjective interpretation. This dependence on manual methods highlights the need for more scalable and automated approaches to taxonomy development.

A notable limitation was the decision not to analyze System IDs for matched data, as these behaved correctly across all test cases. Instead, the analysis for matched data focused on relationships between functional abstractions using Sheet IDs. While this approach provided valuable insights, it overlooked the potential for deeper patterns in consistently passing or failing System IDs. In contrast, non-matching data analysis incorporated System IDs, allowing for a more detailed evaluation of implementation-specific flaws.

Finally, the taxonomy was developed through exploratory analysis without formal validation or verification. This lack of formal validation limits confidence in its completeness and applicability to broader contexts. While the exploratory nature of this work provided foundational insights, the taxonomy requires further refinement and validation to ensure its generalizability and robustness.

8.2. Future Work

A major focus for future work should be the validation of the taxonomy. Due to the exploratory scope of this research, only a limited subset of functional abstractions, Java exceptions, and source codes were analyzed. Expanding the dataset to include more functional abstractions, benchmarks, and testing conditions would ensure the taxonomy’s robustness and broader applicability. Validating the taxonomy against diverse programming contexts and additional benchmarks would confirm its reliability and relevance.

Another essential direction is edge case testing, which emerged as a recurring challenge in the non-matching data analysis. Many mismatches stemmed from failures to address edge cases, such as unusual input formats or boundary conditions. Future research should systematically identify these scenarios and analyze how they affect the functional correctness of LLM-generated code. This would provide deeper insights into improving edge case handling in automated code generation.

System ID-level analysis presents another area of opportunity. While this thesis focused on Sheet ID -level comparisons for matched data, further analysis of System IDs could reveal more meaningful patterns. For example, identifying System IDs that consistently pass or fail across all test cases for a given functional abstraction could help uncover characteristics of reliable or flawed implementations. This, in turn, could provide actionable insights into why certain LLM-generated solutions perform better than others across diverse contexts.

Additionally, addressing translation-related issues between Python and Java benchmarks remains a key area for improvement. Anomalies such as the insertion of empty dictionaries in inputs suggest potential translation losses, either during the Python-to-Java conversion by MultiPL-E or during the data import into LASSO.

Finally, the taxonomy itself could be extended to incorporate additional benchmarks, programming languages, and evaluation metrics. Expanding the taxonomy’s scope to cover a broader dataset would provide a more comprehensive framework for evaluating LLM-generated code across varied contexts. These enhancements could inform practical applications, helping to refine LLMs for software development and testing tasks.

9. Conclusion

The goal of this thesis was to develop a taxonomy for categorizing software responses, providing a deeper understanding of their behavior and functional characteristics. In order to build this taxonomy, the methodology proposed in [30] was adapted.

Data from the HumanEval and MBPP benchmarks, including their Java translations by MultiPL-E, formed the foundation of this study. The dataset, consisting of over 300,000 unique system outputs, was analyzed by categorizing responses into matched and unmatched data. For matched data, relationships such as behavioral equivalence and functional similarity were identified, while unmatched data was classified into errors, exceptions, and valid but diverging outputs.

The findings revealed significant challenges in LLM-generated code, including systematic issues in edge case handling, translation-related anomalies, and logical flaws in implementations. For example, additional empty dictionaries observed in Java exception inputs pointed to potential translation losses during Python-to-Java conversions or data import processes. These anomalies underscored the importance of quality control in benchmark preparation and cross-language adaptation.

By focusing on Sheet IDs for matched data and System IDs for unmatched data, this research demonstrated the potential for analyzing both functional abstraction relationships and implementation-specific flaws. However, the study’s exploratory nature and reliance on subsets of data limited the taxonomy’s scope, making it a preliminary framework rather than a comprehensive model.

Despite its limitations, this thesis provides a foundational taxonomy for categorizing response types in LLM-generated code, offering valuable insights into their functional characteristics and common implementation challenges. The adapted methodology and findings lay the groundwork for future research, including validating and extending the taxonomy to additional benchmarks, programming languages, and testing conditions.

Bibliography

- [1] Ian Arawjo et al. „ChainForge: A Visual Toolkit for Prompt Engineering and LLM Hypothesis Testing“. In: *Proceedings of the CHI Conference on Human Factors in Computing Systems*. 2024, pp. 1–18.
- [2] Jacob Austin et al. *Program Synthesis with Large Language Models*. 2021. arXiv: 2108.07732 [cs.PL]. URL: <https://arxiv.org/abs/2108.07732>.
- [3] Sussy Bayona-Oré et al. „Critical success factors taxonomy for software process deployment“. In: *Software Quality Journal* 22 (2014), pp. 21–48.
- [4] Ricardo Britto, Emilia Mendes, and Claes Wohlin. „A specialized global software engineering taxonomy for effort estimation“. In: *2016 IEEE 11th International Conference on Global Software Engineering (ICGSE)*. IEEE. 2016, pp. 154–163.
- [5] Ricardo Britto, Claes Wohlin, and Emilia Mendes. „An extended global software engineering taxonomy“. In: *Journal of Software Engineering Research and Development* 4 (2016), pp. 1–24.
- [6] Federico Cassano et al. „MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation“. In: *IEEE Transactions on Software Engineering* 49.7 (2023), pp. 3675–3691. DOI: 10.1109/TSE.2023.3267446.
- [7] Mark Chen et al. *Evaluating Large Language Models Trained on Code*. 2021. arXiv: 2107.03374 [cs.LG]. URL: <https://arxiv.org/abs/2107.03374>.
- [8] Google Research Datasets. *Mostly Basic Python Problems (MBPP)*. <https://huggingface.co/datasets/google-research-datasets/mbpp>. Accessed: January 30, 2025. 2021.
- [9] Institute of Electrical and Electronics Engineers. *IEEE Standard Taxonomy for Software Engineering Standards: Approved Dec. 11, 1986 IEEE Standards Board; Approved June 4, 1987 American National Standards Inst.* IEEE, 1987.

-
- [10] Google Research. *Pathways Language Model (PaLM)*. Accessed: January 31, 2025. 2022. URL: <https://ai.googleblog.com/2022/04/pathways-language-model-palm-scaling-to.html>.
 - [11] Bill Hetzel. *The complete guide to software testing*. QED Information Sciences, Inc., 1988.
 - [12] Marcus Kessel and Colin Atkinson. „Promoting open science in test-driven software experiments“. In: *Journal of Systems and Software* 212 (2024), p. 111971.
 - [13] Kashumi Madampe, Rashina Hoda, and John Grundy. „A faceted taxonomy of requirements changes in agile contexts“. In: *IEEE Transactions on Software Engineering* 48.10 (2021), pp. 3737–3752.
 - [14] Emilia Mendes, Steve Counsell, and Nile Mosley. „Towards a taxonomy of hypermedia and web application size metrics“. In: *International Conference on Web Engineering*. Springer. 2005, pp. 110–123.
 - [15] Tom Mens et al. „Towards a taxonomy of software evolution“. In: *Proceedings of the International Workshop on Unanticipated Software Evolution*. 2003.
 - [16] Lincoln Murr, Morgan Grainger, and David Gao. „Testing LLMs on Code Generation with Varying Levels of Prompt Specificity“. In: *arXiv preprint arXiv:2311.07599* (2023).
 - [17] Ana Nunez et al. „Autosafecoder: A multi-agent framework for securing llm code generation through static analysis and fuzz testing“. In: *arXiv preprint arXiv:2409.10737* (2024).
 - [18] nuprl. *MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation*. 2022. URL: <https://huggingface.co/datasets/nuprl/MultiPL-E>.
 - [19] Software Observatorium. *Cell Value Model Documentation*. <https://github.com/SoftwareObservatorium/lasso/blob/develop/doc/internal/CellValue.md>. 2024.
 - [20] OpenAI. *HumanEval: A Benchmark for Evaluating Code Generation from LLMs*. <https://github.com/openai/human-eval>. Accessed: January 30, 2025. 2021.

- [21] OpenAI. *OpenAI Codex: AI-powered code generation*. Accessed: January 31, 2025. 2021. URL: <https://openai.com/research/codex>.
- [22] Pranab Sahoo et al. „A systematic survey of prompt engineering in large language models: Techniques and applications“. In: *arXiv preprint arXiv:2402.07927* (2024).
- [23] Shubhra Kanti Karmaker Santu and Dongji Feng. „Teler: A general taxonomy of llm prompts for benchmarking complex tasks“. In: *arXiv preprint arXiv:2305.11430* (2023).
- [24] Mohamed Shehata, Armin Eberlein, and Abraham O Fapojuwo. „A taxonomy for identifying requirement interactions in software systems“. In: *Computer Networks* 51.2 (2007), pp. 398–425.
- [25] Igor Regis da Silva Simões and Elaine Venson. „Evaluating Source Code Quality with Large Language Models: a comparative study“. In: *Proceedings of the XXIII Brazilian Symposium on Software Quality*. 2024, pp. 103–113.
- [26] Darja Šmite et al. „An empirically based terminology and taxonomy for global software engineering“. In: *Empirical Software Engineering* 19 (2014), pp. 105–153.
- [27] Hamed Taherkhani et al. „Epic: Cost-effective search-based prompt engineering of llms for code generation“. In: *arXiv preprint arXiv:2408.11198* (2024).
- [28] Weixi Tong and Tianyi Zhang. „CodeJudge: Evaluating Code Generation with Large Language Models“. In: *arXiv preprint arXiv:2410.02184* (2024).
- [29] Muhammad Usman, Jürgen Börstler, and Kai Petersen. „An effort estimation taxonomy for agile software development“. In: *International Journal of Software Engineering and Knowledge Engineering* 27.04 (2017), pp. 641–674.
- [30] Muhammad Usman et al. „Taxonomies in software engineering: A systematic mapping study and a revised taxonomy development method“. In: *Information and Software Technology* 85 (2017), pp. 43–59.
- [31] Jules White et al. „A prompt pattern catalog to enhance prompt engineering with chatgpt“. In: *arXiv preprint arXiv:2302.11382* (2023).

-
- [32] Dewu Zheng et al. „How Well Do LLMs Generate Code for Different Application Domains? Benchmark and Evaluation“. In: *arXiv preprint arXiv:2412.18573* (2024).
 - [33] Dewu Zheng et al. „Towards more realistic evaluation of LLM-based code generation: an experimental study and beyond“. In: *arXiv preprint arXiv:2406.06918* (2024).

Appendix

Appendix for Chapter 5

Relationship	Abstraction Clusters
Behaviorally Equivalent	[mbpp.127_multiply_int, mbpp.458_rectangle_area]
Identical	[mbpp.591_swap_List, mbpp.625_swap_List]
Partially Behaviorally Equivalent	[mbpp.786_right_insertion, mbpp.736_left_insertion] [mbpp.790_even_position, mbpp.775_odd_position]
Functionally Similar	[HumanEval.53_add, mbpp.797_sum_in_range] [HumanEval.73_smallest_change, mbpp.296_get_Inv_Count] [mbpp.68_is_Monotonic, mbpp.760_unique_Element] [mbpp.68_is_Monotonic, mbpp.775_odd_position, mbpp.790_even_position] [mbpp.68_is_Monotonic, mbpp.775_odd_position] [mbpp.790_even_position, HumanEval.126_is_sorted]
Contextually Related	[HumanEval.108_count_nums, HumanEval.69_search] [HumanEval.14_all_prefixes, HumanEval.17_parse_music, HumanEval.101_words_string] [HumanEval.27_flip_case, HumanEval.10_make_palindrome, HumanEval.51_remove_vowels, HumanEval.86_anti_shuffle, HumanEval.19_sort_numbers] [HumanEval.83_starts_one_ends, mbpp.592_sum_of_product, HumanEval.55_fib, HumanEval.60_sum_to_n, HumanEval.139_special_factorial]
Contextually Related - Functionally Similar (Broadly Connected)	[HumanEval.109_move_one_ball, mbpp.760_unique_Element, mbpp.19_test_duplicate] [HumanEval.126_is_sorted, HumanEval.109_move_one_ball] [HumanEval.126_is_sorted, mbpp.19_test_duplicate] [HumanEval.57_monotonic, HumanEval.126_is_sorted] [mbpp.281_all_unique, mbpp.760_unique_Element] [mbpp.472_check_Consecutive, mbpp.281_all_unique, HumanEval.126_is_sorted, HumanEval.109_move_one_ball] [mbpp.472_check_Consecutive, mbpp.281_all_unique] [mbpp.751_check_min_heap, HumanEval.126_is_sorted]
Contextually Related - Functionally Similar (Self-Contained)	[HumanEval.104_unique_digits, HumanEval.22_filter_integers, HumanEval.42_incr_list, HumanEval.9_rolling_max, HumanEval.145_order_by_points, HumanEval.88_sort_array, HumanEval.116_sort_array, HumanEval.26_remove_duplicates, HumanEval.30_get_positive, HumanEval.68_pluck, HumanEval.70_strange_sort_list, HumanEval.105_by_length, HumanEval.149_sorted_list_sum] [HumanEval.142_sum_squares, mbpp.798_sum] [HumanEval.16_count_distinct_characters, mbpp.172_count_occurrence, HumanEval.98_count_upper, HumanEval.23_strlen, HumanEval.66_digitSum] [HumanEval.24_largest_divisor, mbpp.295_sum_div] [HumanEval.29_filter_by_prefix, HumanEval.7_filter_by_substring] [HumanEval.43_pairs_sum_to_zero, HumanEval.40_triples_sum_to_zero] [HumanEval.63_fibfib, HumanEval.46_fib4] [HumanEval.74_total_match, HumanEval.58_common] [mbpp.435_last_Digit, mbpp.577_last_Digit_Factorial] [mbpp.454_text_match_wordz, mbpp.643_text_match_wordz_middle]
Unrelated	[HumanEval.106_f, HumanEval.123_get_odd_collatz] [HumanEval.12_longest, HumanEval.128_prod_signs, HumanEval.90_next_smallest] [HumanEval.138_is_equal_to_sum_even, HumanEval.31_is_prime] [HumanEval.31_is_prime, mbpp.600_is_Even] [HumanEval.73_smallest_change, HumanEval.146_specialFilter] [HumanEval.75_is_multiply_prime, mbpp.781_count_divisors] [HumanEval.82_prime_length, HumanEval.134_check_if_last_char_is_a_letter, HumanEval.132_is_nested, HumanEval.124_valid_date] [mbpp.138_is_Sum_of_Powers_of_Two, mbpp.781_count_divisors, mbpp.3_is_not_prime, HumanEval.138_is_equal_to_sum_even]
Unrelated Completely	[HumanEval.162_string_to_md5, HumanEval.111_histogram] [HumanEval.75_is_multiply_prime, mbpp.803_is_perfect_square, mbpp.72_diff_Square, mbpp.406_find_Parity]

Table A1.: Categorization of abstraction clusters by behavioral relationships

Appendix for Chapter 6

A.1. Appendix for Section 6.1

```
1 public static ArrayList<Integer> sortEven(ArrayList<Integer> l) {
2     ArrayList<Integer> ret = new ArrayList<>();
3     ArrayList<Integer> l_even = new ArrayList<>();
4     ArrayList<Integer> l_odd = new ArrayList<>();
5
6     // Splitting even and odd indexed elements
7     for (int i = 0; i < l.size(); i++) {
8         if (i % 2 == 0) {
9             l_even.add(l.get(i));
10        } else {
11            l_odd.add(l.get(i));
12        }
13    }
14
15    Collections.sort(l_even); // Sorting only even indexed elements
16
17    int i = 0;
18    for (Integer n : l_even) {
19        ret.add(n);
20        ret.add(l_odd.get(i)); // Fails if l_even and l_odd sizes do not
                               // match
21        i++;
22    }
23    return ret;
24 }
```

Listing 1: Incorrect Implementation: "HumanEval_37_sort_even"

```
1 public static ArrayList<Long> sortThird(ArrayList<Long> l) {
2     ArrayList<Long> l2 = new ArrayList<>();
3     for (int i = 0; i < l.size(); i++) {
4         if (i % 3 == 0) {
5             ArrayList<Long> l3 = new ArrayList<>();
6             l3.add(l.get(i));
7             l3.add(l.get(i + 1)); // Fails if i + 1 is out of bounds
8             l3.add(l.get(i + 2)); // Fails if i + 2 is out of bounds
9             Collections.sort(l3);
10            l2.add(l3.get(0));
11            l2.add(l3.get(1));
12            l2.add(l3.get(2));
13        } else {
14            l2.add(l.get(i));
15        }
16    }
17    return l2;
18 }
```

Listing 2: Incorrect Implementation: "HumanEval_33_sort_third"

```

1 public static boolean correctBracketing(String brackets) {
2     Stack<Character> stack = new Stack<>();
3     for (int i = 0; i < brackets.length(); i++) {
4         char c = brackets.charAt(i);
5         if (Character.isDigit(c)) {
6             while (Character.isDigit(brackets.charAt(i + 1))) {
7                 i++;
8             }
9             stack.pop();
10        } else {
11            if (brackets.charAt(i + 1) != '$<$') {
12                while (!stack.empty()) {
13                    char top = stack.pop();
14                    if (top == '$<$') break;
15                }
16                stack.push(c);
17            }
18        }
19    }
20    return stack.empty();
21 }

```

Listing 3: Incorrect Implementation: "HumanEval_56_correct_bracketing"

```

1 public static Pair<String, Boolean> reverseDelete(String s, String c) {
2     List<Character> list = new ArrayList<>();
3     for (int i = 0; i < s.length(); i++) {
4         list.add(s.charAt(i));
5     }
6     for (int i = 0; i < c.length(); i++) {
7         list.remove((int) c.charAt(i));
8         // Fails when c.charAt(i) exceeds valid index range
9     }
10    StringBuilder sb = new StringBuilder();
11    for (int i = 0; i < list.size(); i++) {
12        sb.append(list.get(i));
13    }
14    return new Pair<>(sb.toString(), true);
15 }

```

Listing 4: Incorrect Implementation: "HumanEval_112_reverse_delete"

```

1 public static ArrayList<Long> sortArray(ArrayList<Long> arr) {
2     ArrayList<Long> result = new ArrayList<Long>();
3     if (arr.size() == 0) {
4         return result; // Handles empty array
5     }
6     int[] count = new int[32];
7     for (int i = 0; i < arr.size(); i++) {
8         count[Long.numberOfLeadingZeros(arr.get(i))]++;
9         // Incorrectly iterates without properly sorting binary
10        // representations
11    }
12    for (int i = 0; i < count.length; i++) {
13        for (int j = 0; j < count[i]; j++) {
14            result.add(arr.get(i));
15            // Fails when accessing out-of-bounds indices
16        }
17    }
18    return result;
19 }

```

Listing 5: Incorrect Implementation: "HumanEval_116_sort_array"

```
1 public static long startsOneEnds(long n) {
2     if (n == 1) {
3         return 9;
4     }
5     long res = 0;
6     long base = 1;
7     for (int i = 1; i < n; i++) {
8         base *= 10;
9     }
10    res += 9 * base;
11    long num = 1;
12    for (int i = 1; i < n; i++) {
13        num *= 10;
14    }
15    res += 9 * num;
16    res += startsOneEnds(n - 2);
17    // Recursive call causes stack overflow for larger values of n
18    return res;
19 }
```

Listing 6: Incorrect Implementation: "HumanEval_83_starts_one_ends"

```
1 public static ArrayList<Long> maximum(ArrayList<Long> arr, long k) {
2     ArrayList<Long> result = new ArrayList<>();
3     long maxValue = arr.get(0);
4     long maxIndex = -1;
5     for (int i = 1; i < arr.size(); i++) {
6         if (arr.get(i) > maxValue) {
7             maxValue = arr.get(i);
8             maxIndex = i;
9         }
10    }
11    result.add(maxValue);
12    if (maxIndex + 1 < arr.size()) {
13        ArrayList<Long> subResult = maximum(arr, k - 1);
14        // Recursive call without effectively reducing problem size
15        for (int i = 0; i < subResult.size(); i++) {
16            result.add(subResult.get(i));
17        }
18    }
19    return result;
20 }
```

Listing 7: Incorrect Implementation: "HumanEval_120_maximum"

B.2. Appendix for Section 6.2

```

1 class Problem {
2     public static long chooseNum(long x, long y) {
3         long even = x;
4         long odd = y;
5         long biggest = -1;
6         while (even < odd) {
7             long mid = (even + odd) / 2;
8             if (mid % 2 == 0) {
9                 even = mid;
10            } else {
11                odd = mid;
12            }
13            // No guarantee that even and odd converge, leading to an
              infinite loop
14        }
15        return biggest;
16    }
17 }

```

Listing 8: Infinite Loop in "HumanEval_102_choose_num"

```

1 class Problem {
2     public static long chooseNum(long x, long y) {
3         long even = x;
4         long odd = y;
5         long biggest = -1;
6         while (even < odd) {
7             biggest = even; // Redundant assignment
8             even = even + 2; // Incremental updates without direct progress
9             if (even > odd) {
10                even = even - 2; // Undoing previous adjustments
11                odd = odd + 2; // Additional unnecessary operations
12            }
13        }
14        return biggest;
15    }
16 }

```

Listing 9: Redundant Operations in "HumanEval_102_choose_num"

```

1 class Problem {
2     public static long chooseNum(long x, long y) {
3         if (x > y) {
4             // No explicit handling for invalid ranges
5         }
6         long even = x;
7         long odd = y;
8         while (even < odd) {
9             if (even % 2 == 0) {
10                return even; // Incorrectly assumes the first even number
              is the largest
11            }
12        }
13        return -1; // Fails to handle cases with no even numbers
14    }
15 }

```

Listing 10: Edge Case Failures in "HumanEval_102_choose_num"

```
1 class Problem {
2     public static long chooseNum(long x, long y) {
3         long even = x;
4         long odd = y;
5         long biggest = even;
6         long smallest = odd;
7         while (even < odd) {
8             biggest = even; // First loop adjusts variables
9             smallest = odd;
10            if (biggest % 2 == 0) {
11                even = even + 1;
12            } else {
13                odd = odd + 1;
14            }
15        }
16        while (even > odd) {
17            biggest = even; // Second loop reverses previous adjustments
18            smallest = odd;
19            if (even % 2 == 0) {
20                even = even - 1;
21            } else {
22                odd = odd - 1;
23            }
24        }
25        return biggest;
26    }
27 }
```

Listing 11: Overly Complex Logic in "HumanEval_102_choose_num"

```
1 class Problem {
2     public static long chooseNum(long x, long y) {
3         long even = x;
4         long odd = y;
5         while (even < odd) {
6             long mid = (even + odd) / 2;
7             if (mid % 2 == 0) {
8                 even = mid; // Adjusts even but may cause oscillation
9             } else {
10                odd = mid; // Adjusts odd but does not reduce the range
11                           effectively
12            }
13        }
14        return even; // May fail if oscillation prevents loop exit
15    }
16 }
```

Listing 12: Inconsistent Mid-Point Logic in "HumanEval_102_choose_num"

C.3. Appendix for Section 6.7

```

1 class Problem {
2     public static ArrayList<Long> makeAPile(long n) {
3         ArrayList<Long> result = new ArrayList<>();
4         while (n > 0) { // The loop does not have a valid stopping
5             condition
6             result.add(n);
7             n = (n % 2 == 0) ? n + 1 : n + 2; // Incorrect update leads to
8             uncontrolled growth
9         }
10    }
11 }

```

Listing 13: Infinite loop in "HumanEval_100_make_a_pile"

```

1 class Problem {
2     public static ArrayList<Long> eat(long number, long need, long
3     remaining) {
4         ArrayList<Long> result = new ArrayList<>();
5         while (remaining > 0) { // No validation on whether 'need' is
6             achievable
7             number += need;
8             remaining -= need;
9             result.add(number);
10        }
11    }
12 }

```

Listing 14: Unvalidated input in "HumanEval_159_eat"

```

1 class Problem {
2     public static ArrayList<Long> makeAPile(long n) {
3         ArrayList<Long> result = new ArrayList<>();
4         result.add(n);
5         long next = n;
6         while (next < 10000000000000000000L) { // Arbitrary large limit
7             if (n % 2 == 0) {
8                 next = next + 2;
9             } else {
10                next = next + 1;
11            }
12            result.add(next);
13            n = next;
14        }
15        return result;
16    }
17 }

```

Listing 15: Inefficient logic in "HumanEval_100_make_a_pile"

```

1 class Problem {
2     public static ArrayList<Long> eat(long number, long need, long
3     remaining) {
4         ArrayList<Long> result = new ArrayList<>();
5         long carrotsLeft = 0;
6         while (carrotsLeft < need) { // Incorrect stopping condition
7             carrotsLeft += number;
8             number -= need;
9             result.add(carrotsLeft);
10        }
11        return result;
12    }
13 }

```

Listing 16: Incorrect condition handling in "HumanEval_159_eat"

```
1 class Problem {
2     public static ArrayList<Long> makeAPile(long n) {
3         ArrayList<Long> result = new ArrayList<>();
4         while (n > 0) {
5             result.add(n);
6             n = n + 2; // Inefficient increment
7             if (result.size() > 100000) { // Large output size
8                 break;
9             }
10        }
11        return result;
12    }
13 }
```

Listing 17: Memory constraints in "HumanEval_100_make_a_pile"

```
1 class Problem {
2     public static ArrayList<Long> makeAPile(long n, Map<String, String>
3         extraInput) {
4         // Unexpected extra input parameter
5         ArrayList<Long> result = new ArrayList<>();
6         while (n > 0) {
7             result.add(n);
8             n = (n % 2 == 0) ? n + 1 : n + 2;
9         }
10        return result;
11    }
12 }
```

Listing 18: Translation error in "HumanEval_100_make_a_pile"

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinn-gemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Aachen, January 31, 2025

Bilegjargal Battulga

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Aachen, January 31, 2025

Bilegjargal Battulga